NoSQL

APACHE
HBASE
Cassandra
mongoDB
riak
redis
HYPERTABLE INC
amazon
DynamoDB
MEMCACHED
Neo4j
CouchDB
relax
ORACLE
NOSQL DATABASE

Представлюсь

Антон Пигусов

Команда Bonjour, ex-backend WebServices, flow публикации

Выпускник 13-й школы программистов

О чем будет лекция

- Что такое NoSQL
- Чем отличается от ”SQL”
- Какие бывают NoSQL базы данных
- CAP-теорема
- ACID vs BASE
- ***Перерыв
- Scylla
- Redis
- Clickhouse
- Hadoop

Чего не будет на лекции

- Глубокого погружения в алгоритмы репликации и шардирования
- Детального разбора проблем в распределенных базах данных
- Настройки и разворачивания баз данных
- Использования клиентов для работы из java

Вопросы

- В выделенное время после каждой секции
- Лучше голосом
- В ходе рассказа буду задавать вопросы вам

Краткая справка

NoSQL == ”Not only SQL”

Впервые термин озвучен в 1998 Карло Строцци (*Strozzi NoSQL*), но активно использоваться стал после 2009-го года, когда концепцию начали развивать бигтехи

NoSQL получили развитие вследствие невозможности обрабатывать огромные объемы данных с учетом ограничений реляционных баз

А разве реляционных баз не достаточно?

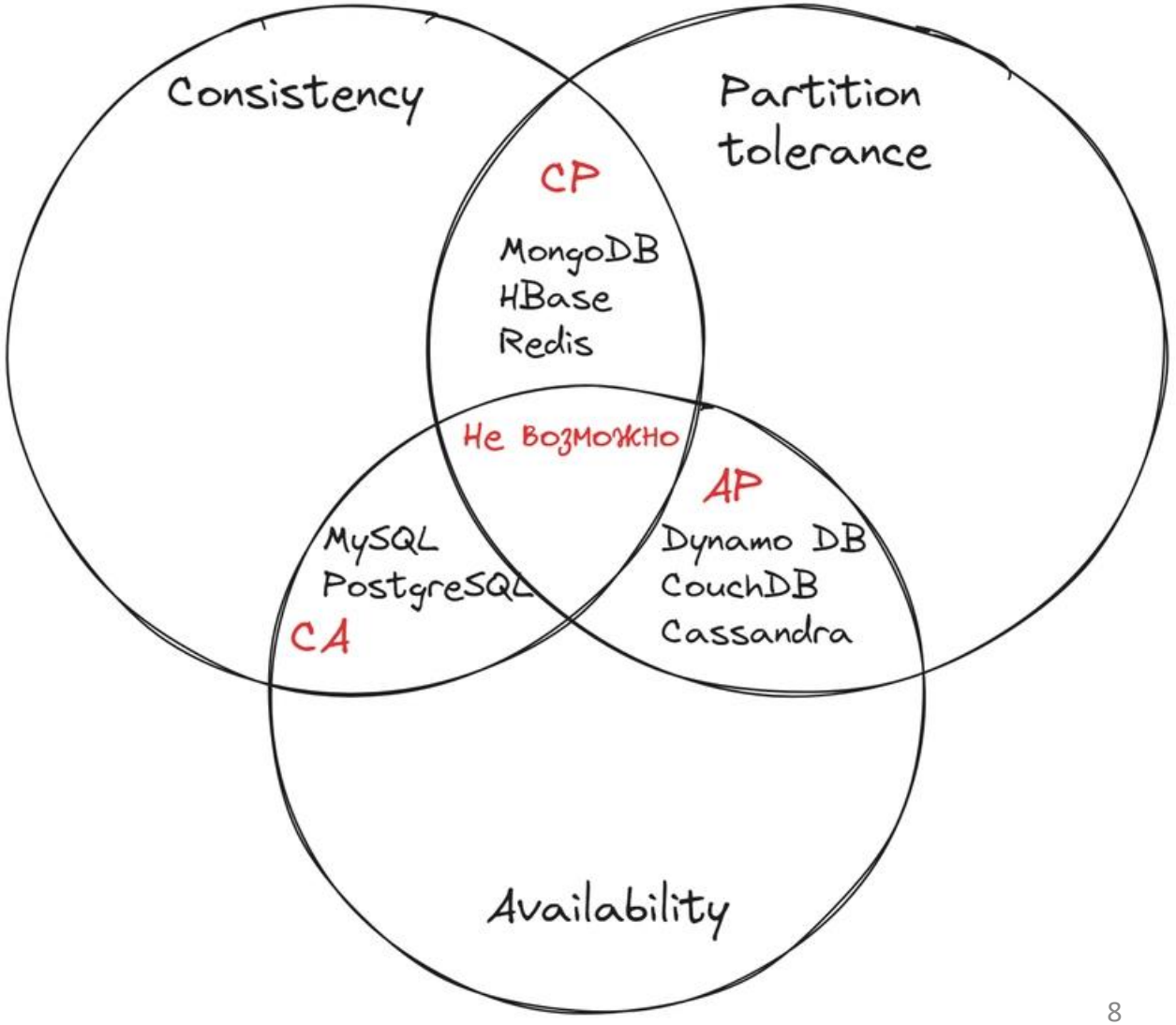
Хранить все данные в реляционных базах не представляется возможным

А разве реляционных баз не достаточно?

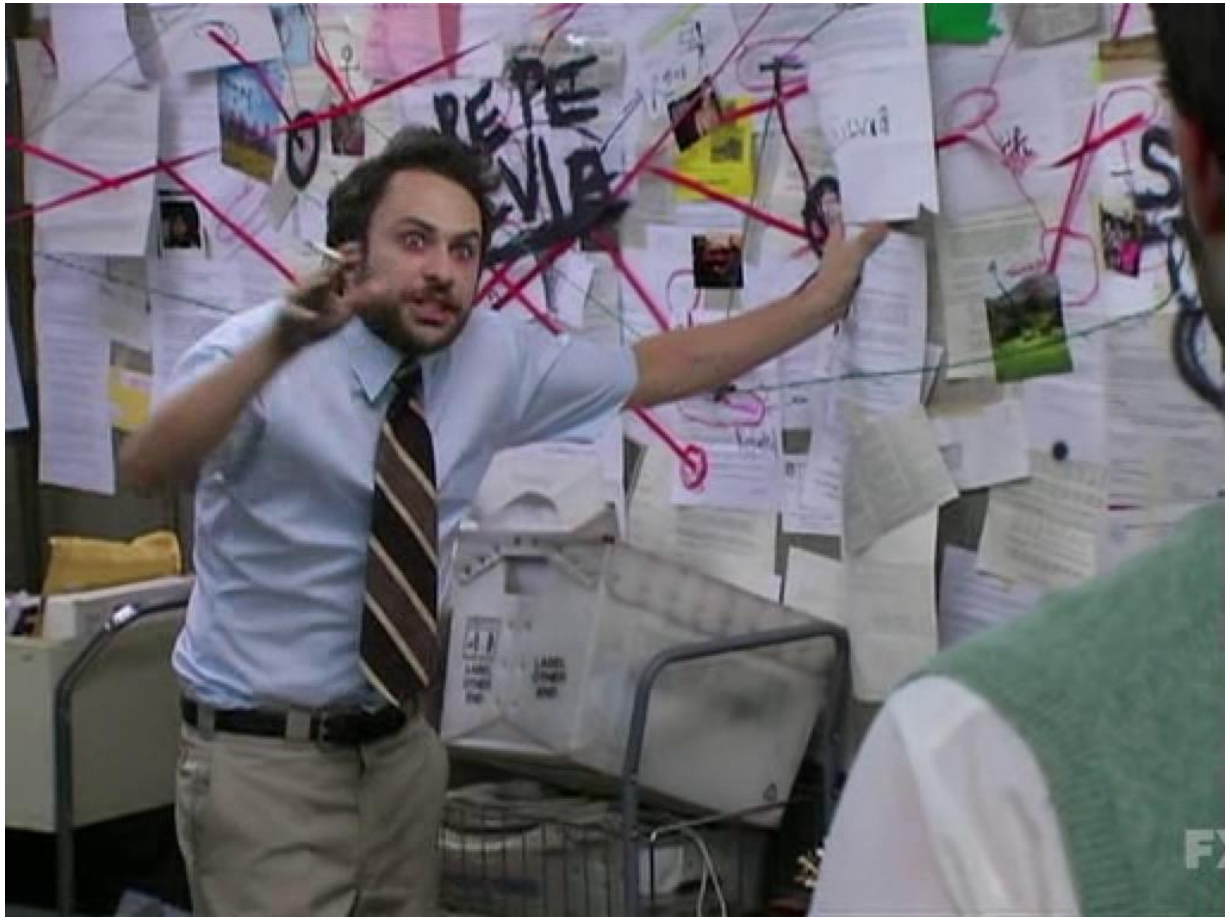
CAP-теорема гласит:

В условиях сетевого разделения (partition) невозможно одновременно обеспечить все три следующих свойства

C — Consistency (Согласованность)
A — Availability (Доступность)
P — Partition tolerance (Устойчивость к сетевым разделениям)

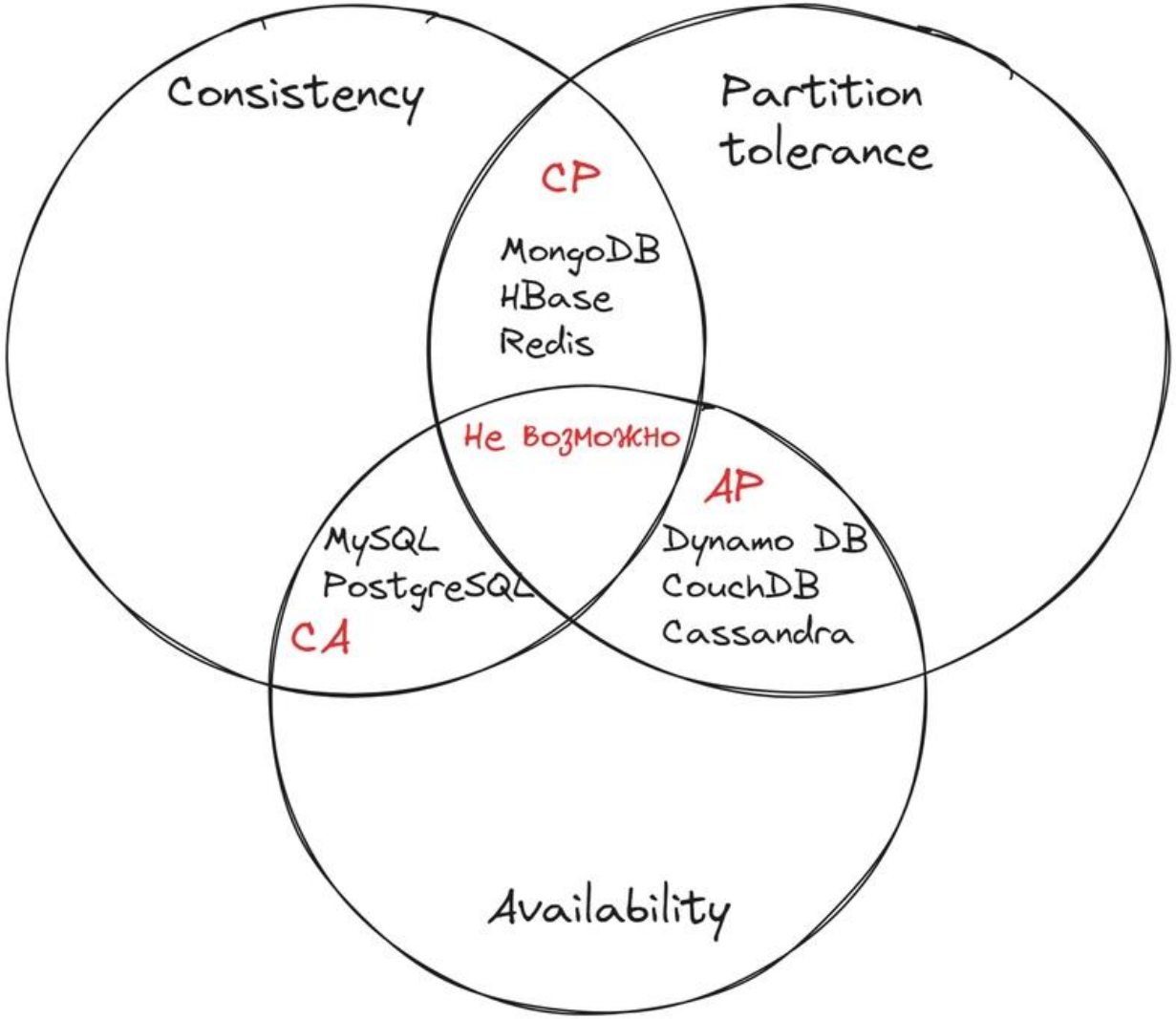


Не все так сложно, как кажется



CAP – варианты

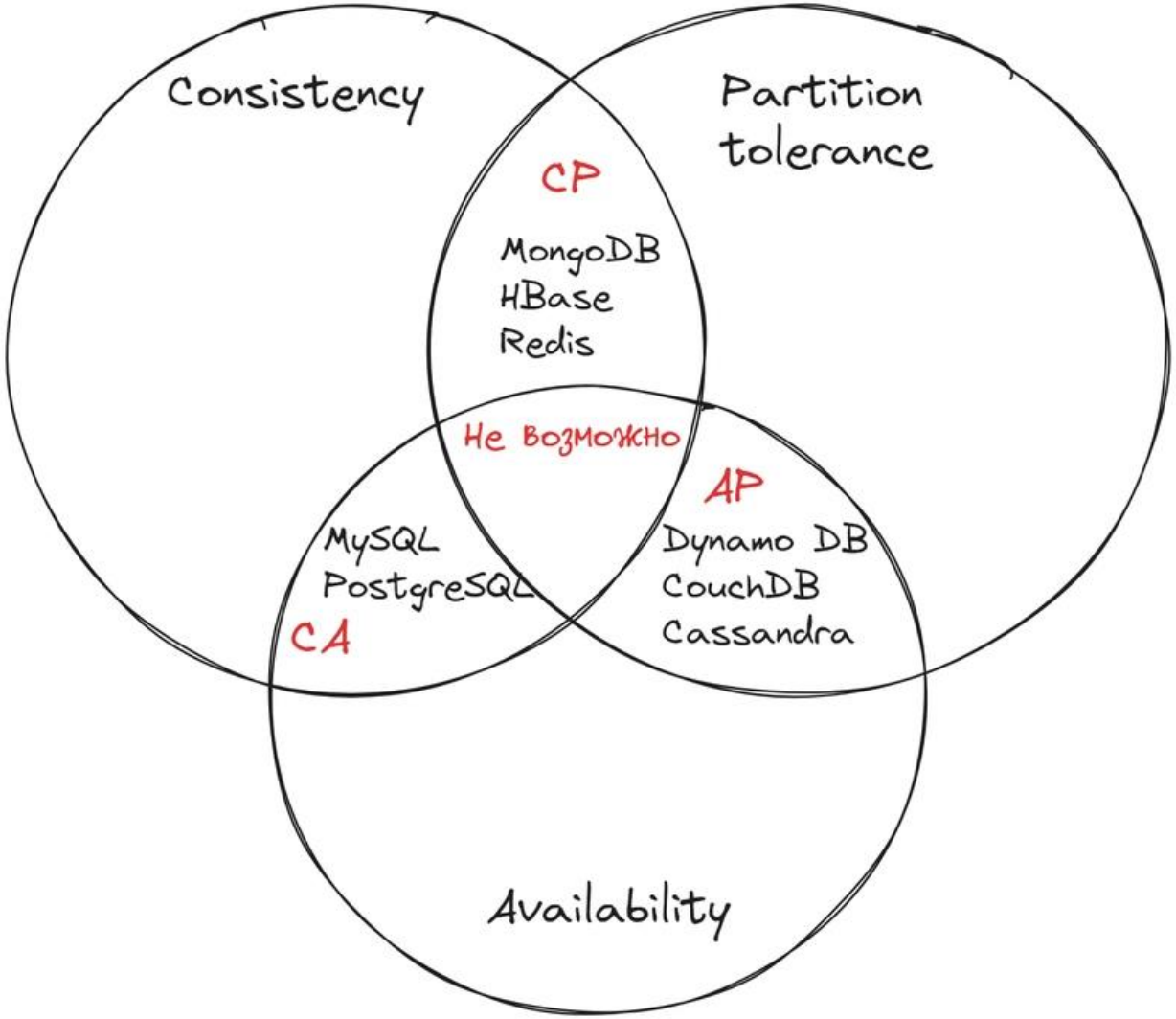
- CA – традиционные реляционные базы данных. Согласованность и доступность только при отсутствии сетевых сбоев



CAP – варианты

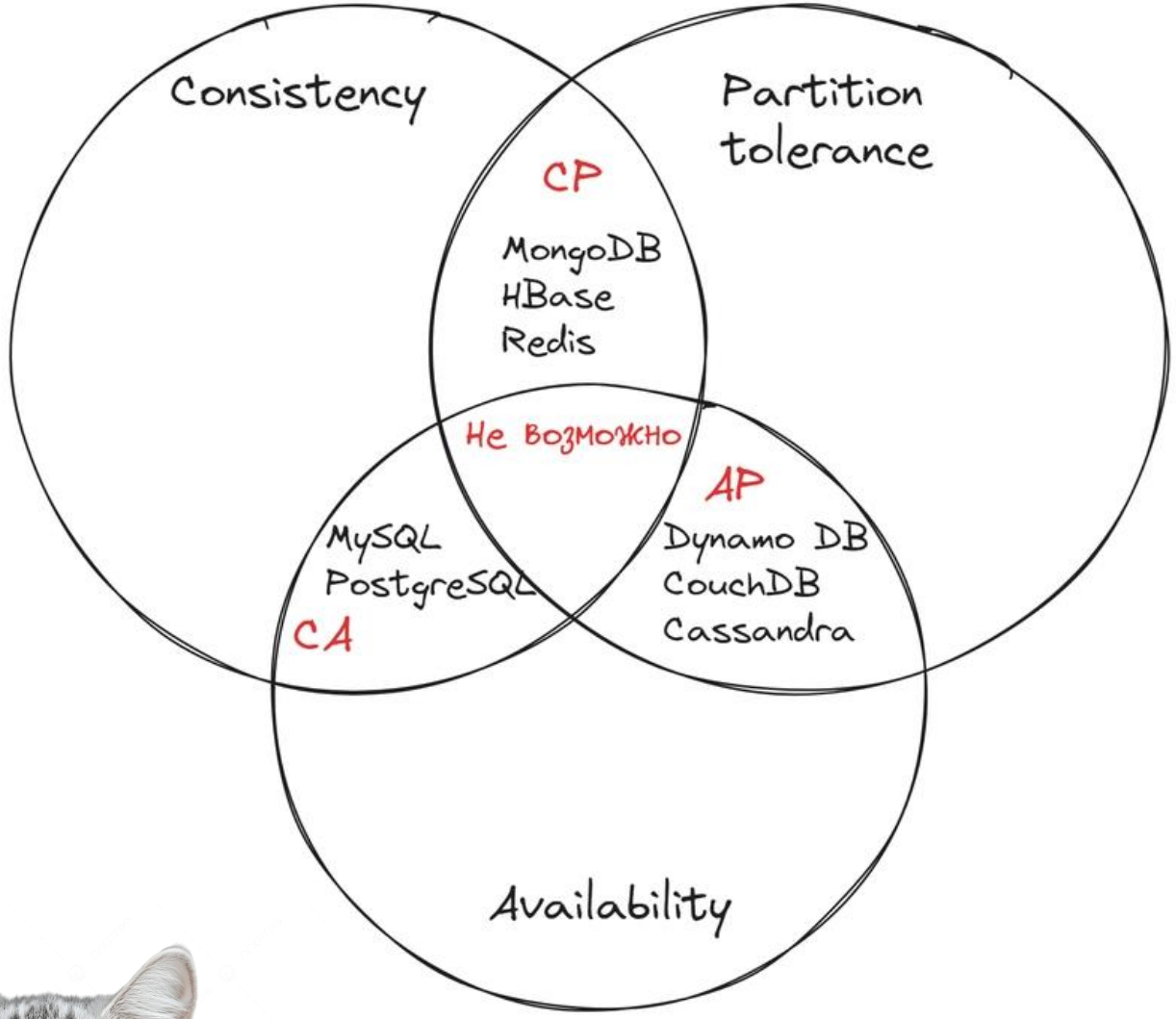
- CA – традиционные реляционные базы данных. Согласованность и доступность только при отсутствии сетевых сбоев

- CP – данные согласованы, но на время сетевого сбоя сервис недоступен



CAP – варианты

- CA – традиционные реляционные базы данных. Согласованность и доступность только при отсутствии сетевых сбоев
- CP – данные согласованы, но на время сетевого сбоя сервис недоступен
- AP – данные могут быть временно не согласованы при постоянно доступном сервисе

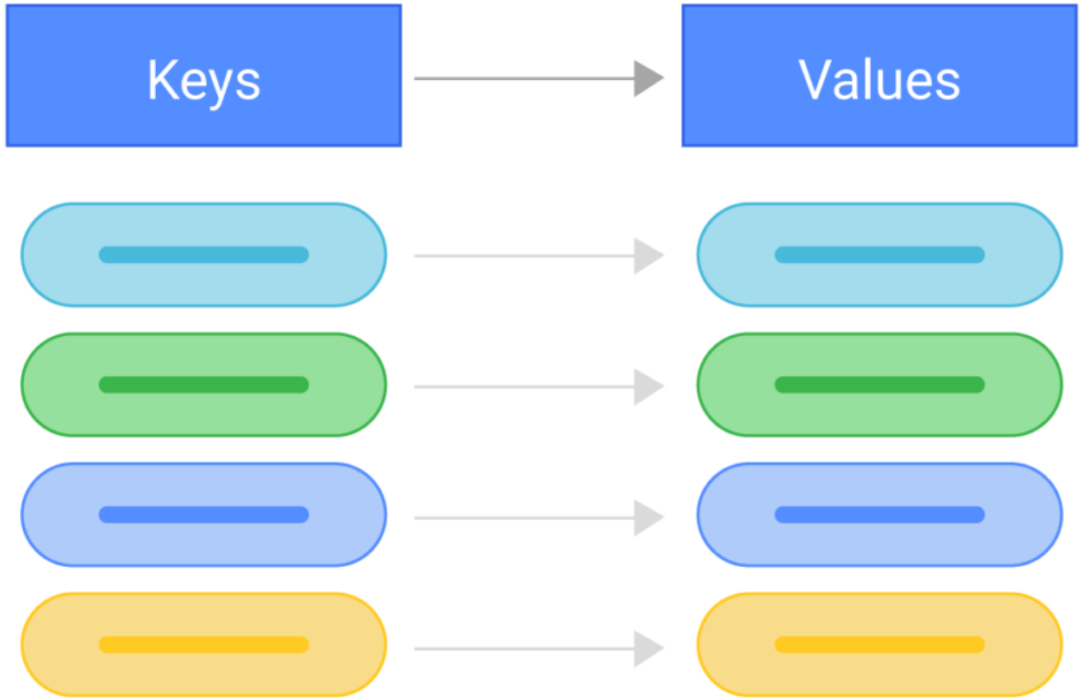


Основные особенности NoSQL

- Большой выбор баз данных в зависимости от задачи

Какие бывают NoSQL базы

- Key-Value (Redis, DynamoDB) – быстрый доступ



Какие бывают NoSQL базы

- Document (MongoDB, Couchbase) – гибкость

Document 1

```
{
  "id": "1",
  “name": "John Smith",
  "isActive": true,
  "dob": "1964-30-08"
}
```

Document 2

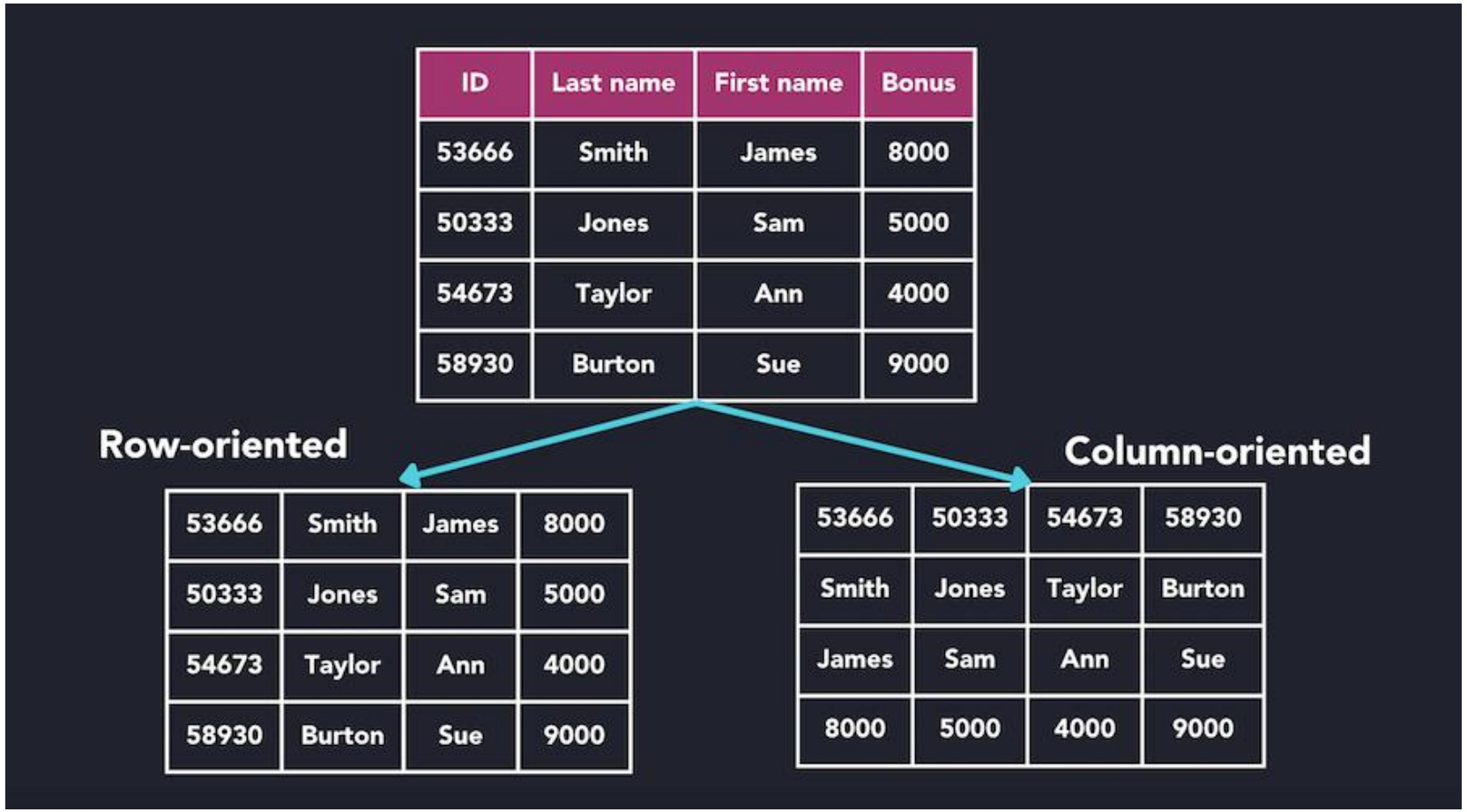
```
{
  "id": "2",
  “fullName": "Sarah Jones",
  “isActive": false,
  "dob": "2002-02-18"
}
```

Document 3

```
{
  "id": "3",
  “fullName":
  {
    "first": "Adam",
    "last": "Stark"
  },
  "isActive": true,
  "dob": "2015-04-19"
}
```

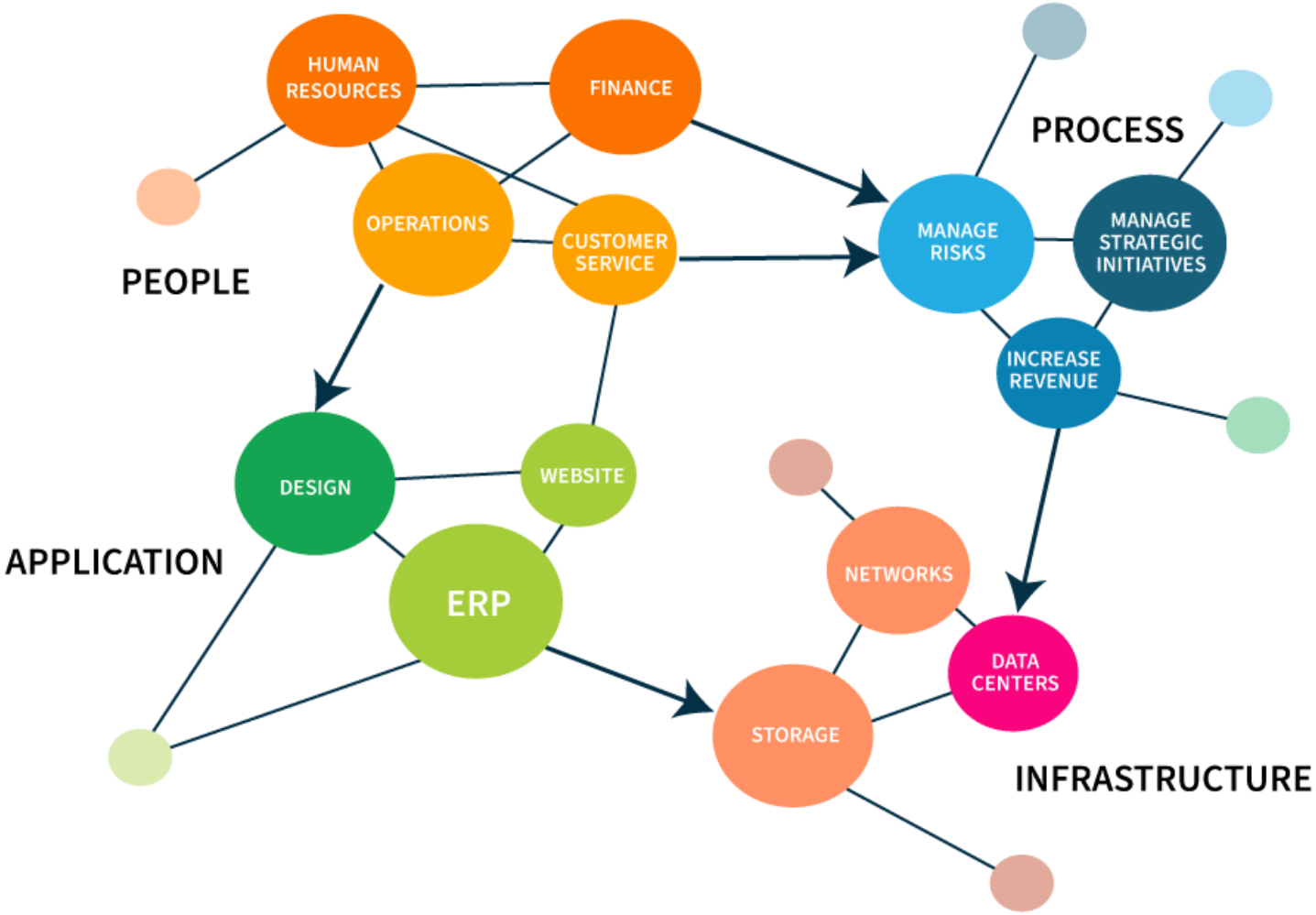
Какие бывают NoSQL базы

- Column (Cassandra, ScyllaDB, Hbase, ClickHouse) – сжатие данных



Какие бывают NoSQL базы

- Graph (Neo4j, Amazon Neptune) – хранение ManyToMany



Какие бывают NoSQL базы

- Time-series / Analytical (ClickHouse, InfluxDB) – тяжелые аналитические запросы

Основные особенности NoSQL

- Большой выбор баз данных в зависимости от задачи
- Гибкая схема данных

А что значит гибкая схема?

Все данные можно вставить в 1 таблицу:

```
// Пользователь 1 — только основные данные
{
    "_id": 1,
    "name": "Анна",
    "email": "anna@example.com"
}

// Пользователь 3 — со списком телефонов
{
    "_id": 3,
    "name": "Алексей",
    "email": "alexei@example.com",
    "phones": ["+79001234567", "+79007654321"],
}
```

```
// Пользователь 2 — с дополнительными полями
{
    "_id": 2,
    "name": "Борис",
    "email": "boris@example.com",
    "age": 30,
    "address": {
        "city": "Москва",
        "street": "Тверская"
    },
    ”newsLetter": true,
}
```

Плюсы

Плюсы

- Не нужно заранее проектировать схему

Плюсы

- Не нужно заранее проектировать схему
- При изменении полей не нужно заниматься миграцией

Плюсы

- Не нужно заранее проектировать схему
- При изменении полей не нужно заниматься миграцией
- Проще тестировать

Плюсы

- Не нужно заранее проектировать схему
- При изменении полей не нужно заниматься миграцией
- Проще тестировать
- Можно хранить любые данные

Плюсы

- Не нужно заранее проектировать схему
- При изменении полей не нужно заниматься миграцией
- Проще тестировать
- Можно хранить любые данные
- Проще масштабировать

Минусы

Минусы

- Обработка несоответствия данных на уровне приложения

Минусы

- Обработка несоответствия данных на уровне приложения
- Можно потерять обратную совместимость

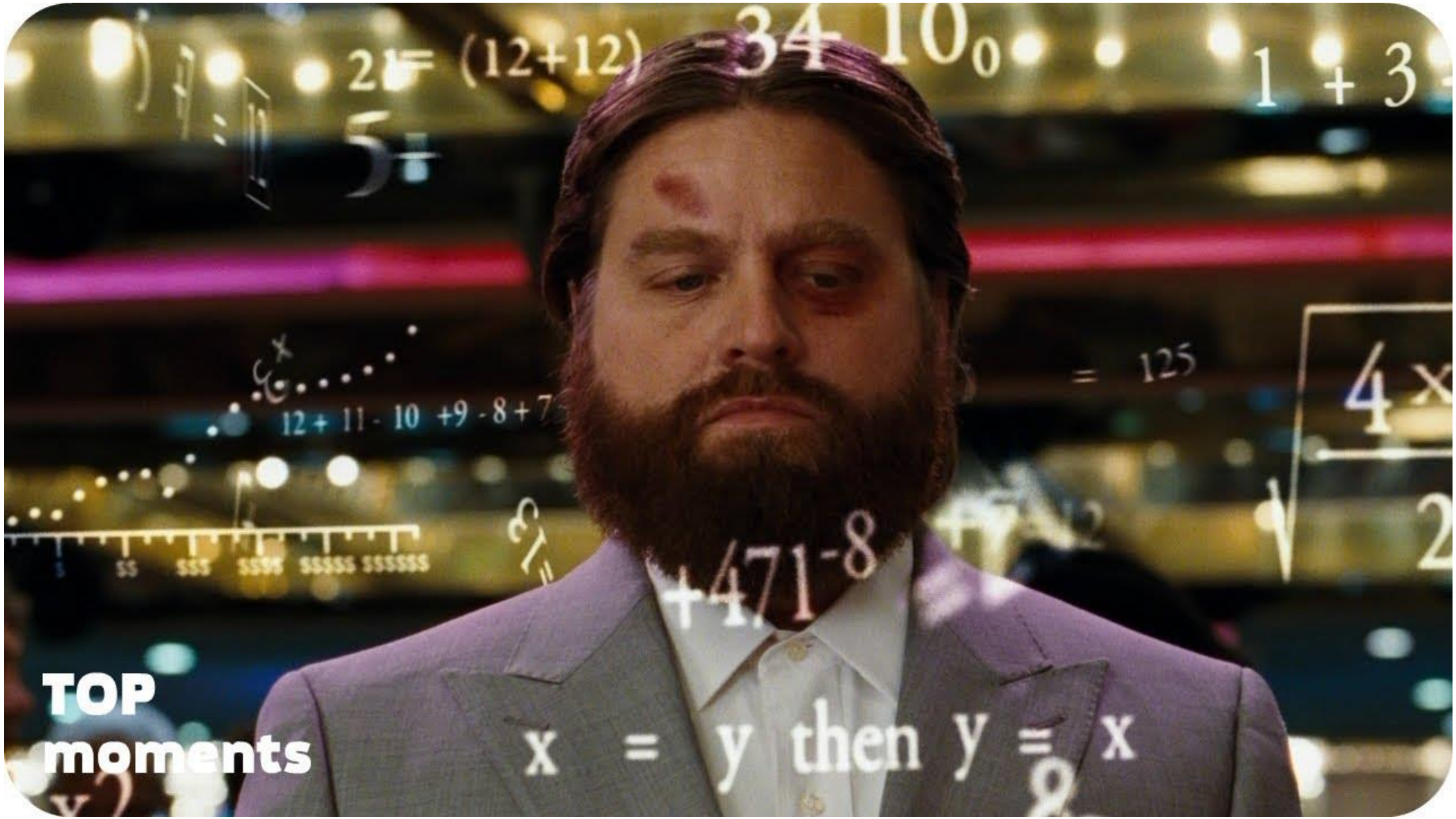
Минусы

- Обработка несоответствия данных на уровне приложения
- Можно потерять обратную совместимость
- Данные плохо структурированы

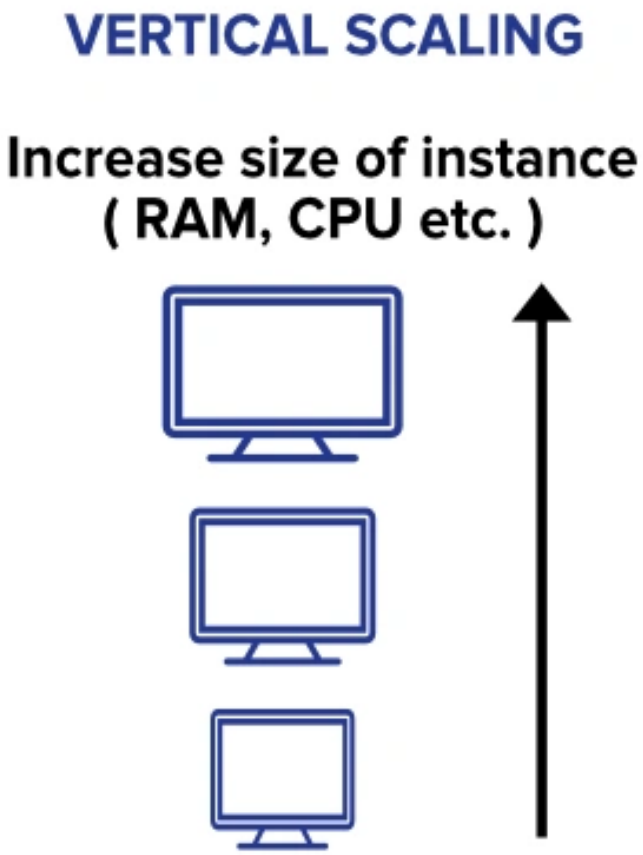
Основные особенности NoSQL

- Большой выбор баз данных в зависимости от задачи
- Гибкая схема данных
- Высокая масштабируемость

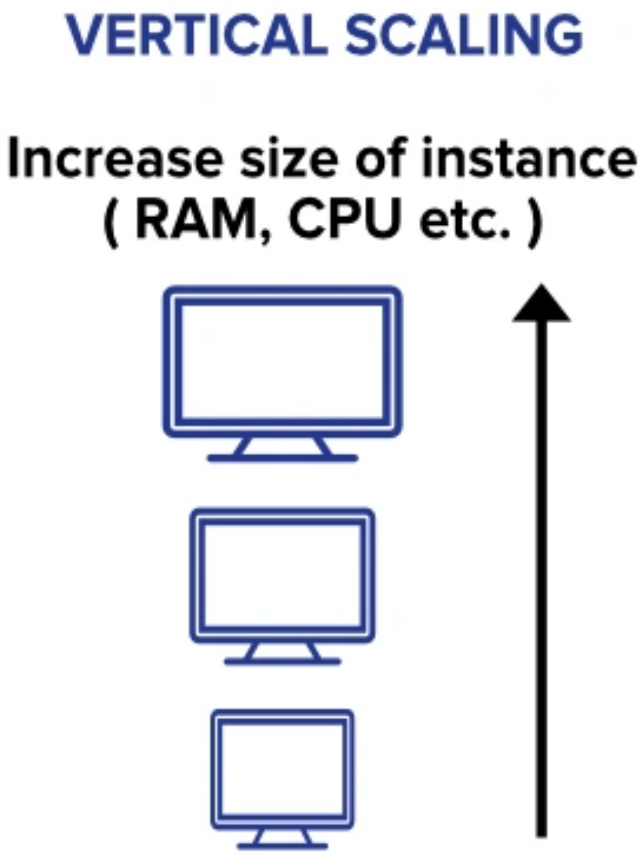
Как обеспечить масштабируемость?

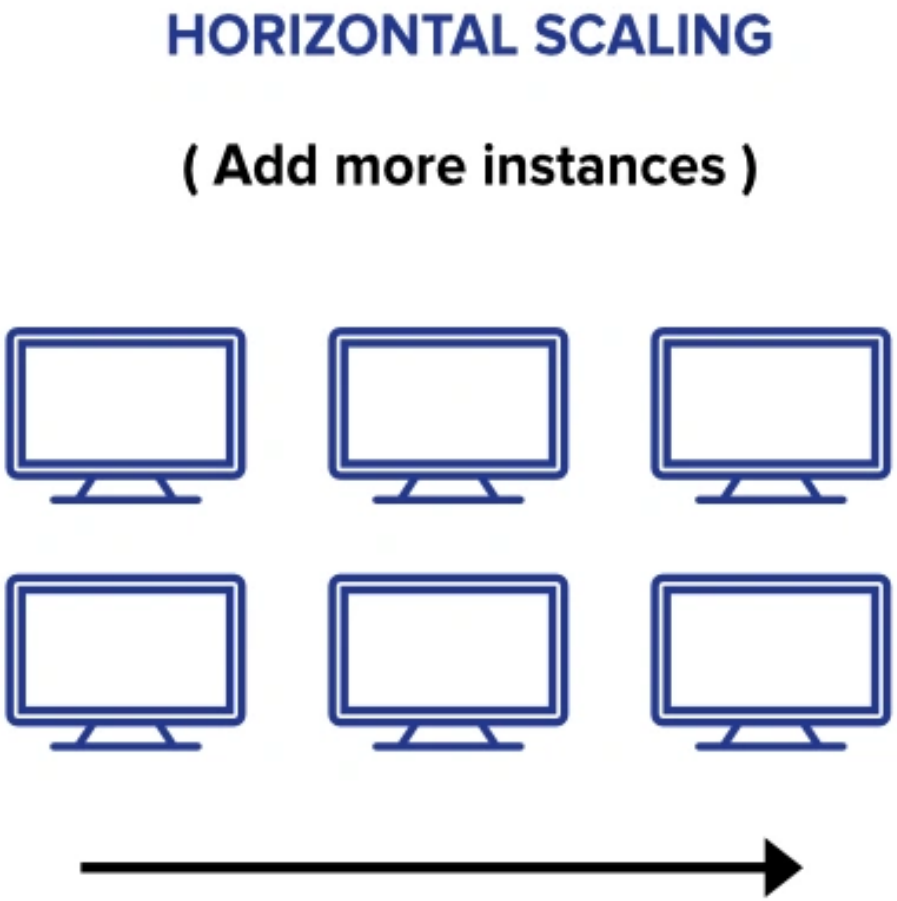


Как обеспечить масштабируемость?



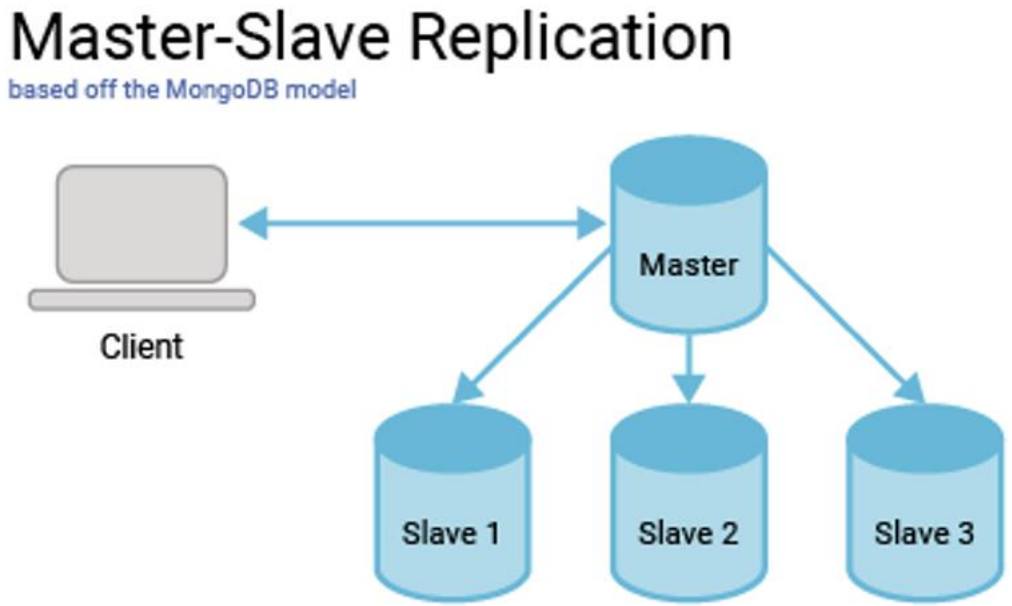
Как обеспечить масштабируемость?



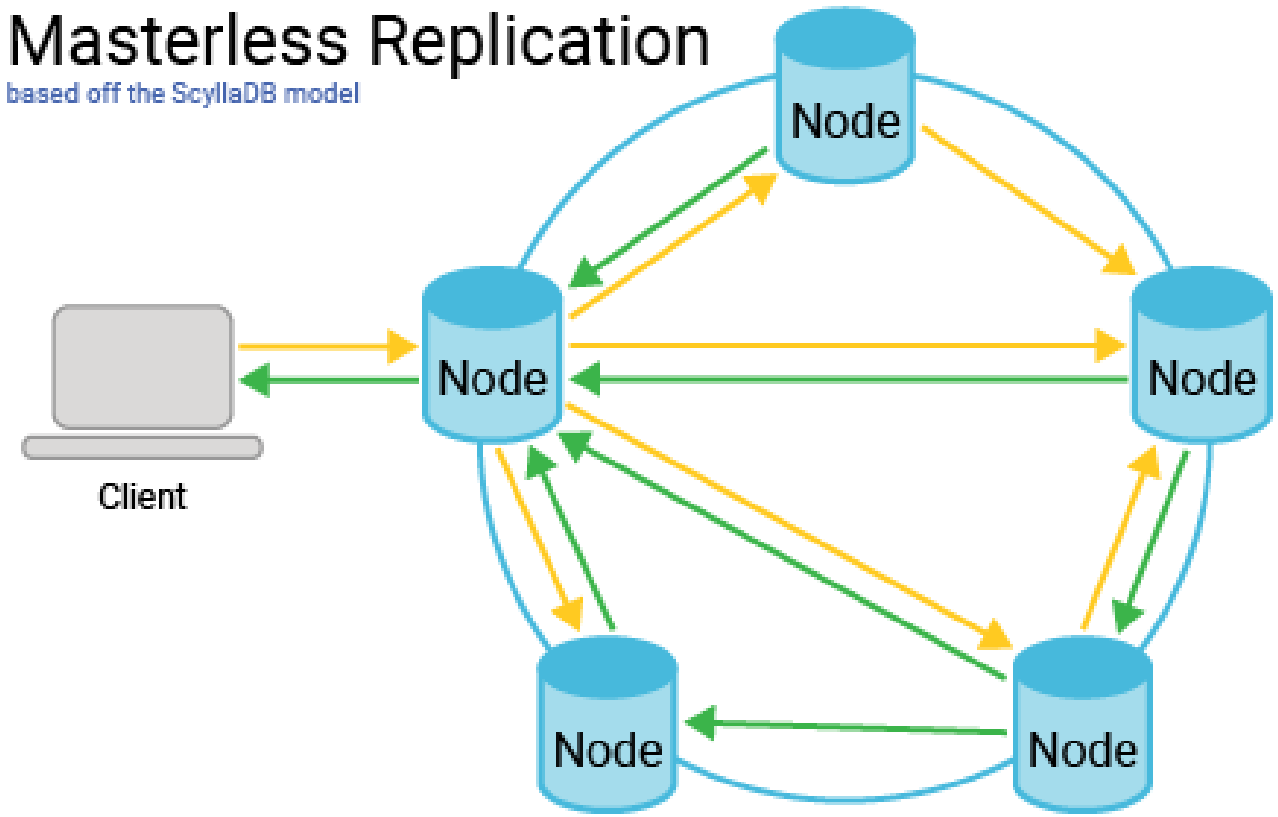


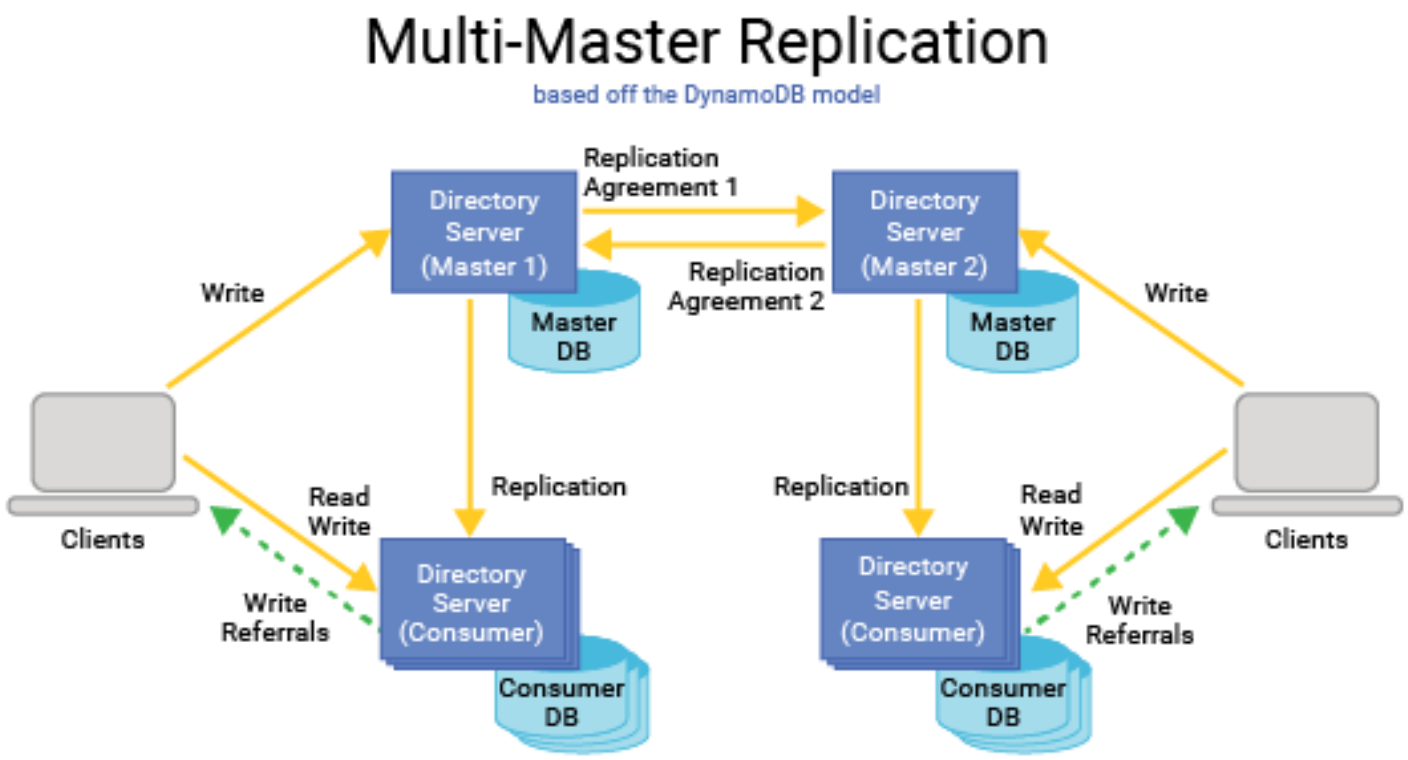
Репликация (и немного про архитектуру)

Копирование одних и тех же данных на несколько узлов (реплик) для обеспечения **отказоустойчивости** и **доступности**



Когда хочется быстрее масштабировать и обеспечить бОльшую отказоустойчивость





Если нужно обеспечить доступность в нескольких ЦОД-ах (например, географически распределенных)

Плюсы

Плюсы

- **Отказоустойчивость**: данные не теряются, если упал один сервер

Плюсы

- **Отказоустойчивость**: данные не теряются, если упал один сервер
- **Чтение с реплик**: можно распределять нагрузку чтения между репликами

Плюсы

- **Отказоустойчивость**: данные не теряются, если упал один сервер
- **Чтение с реплик**: можно распределять нагрузку чтения между репликами
- **Географическая близость**: реплики можно размещать в разных регионах → меньше задержка для пользователей

Минусы

Минусы

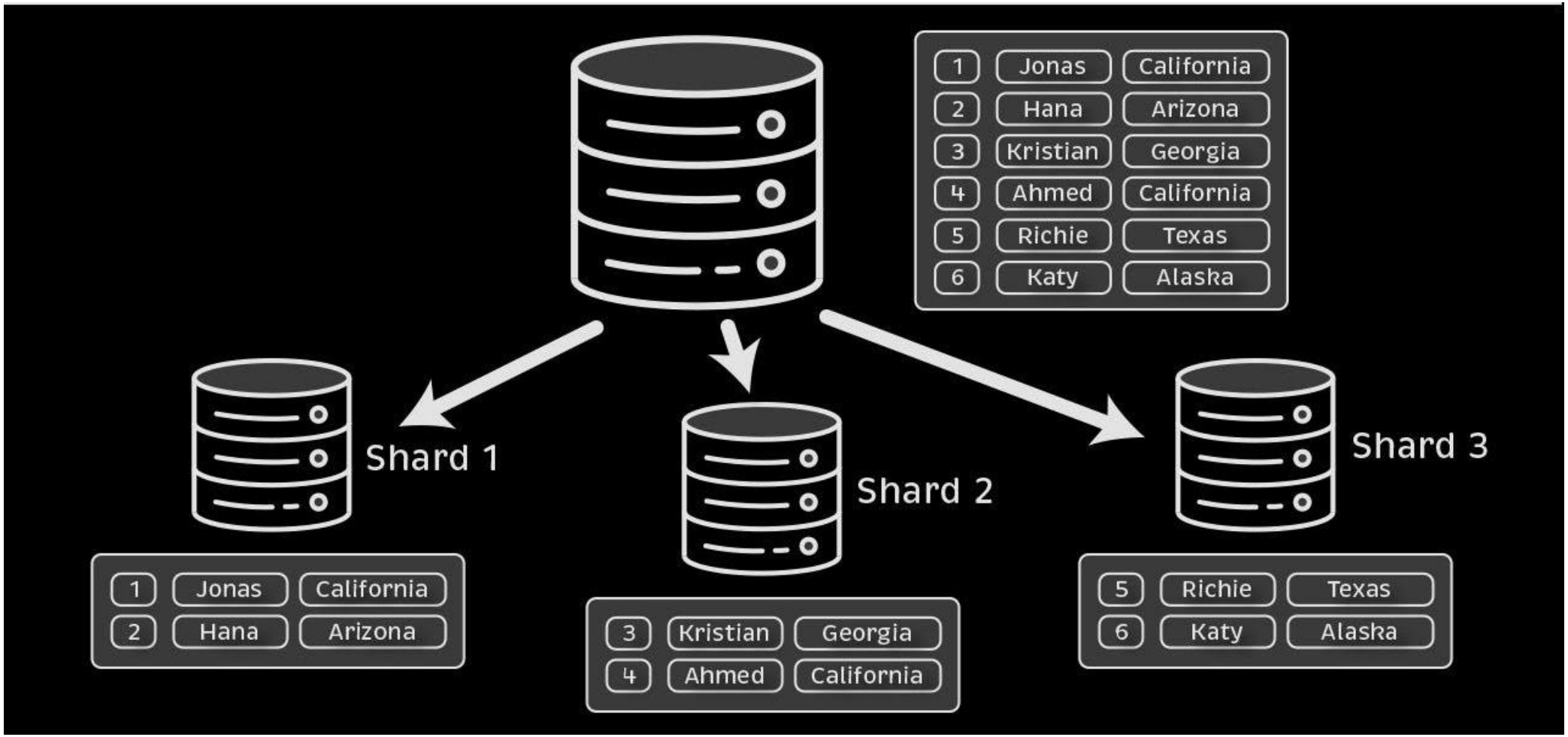
- **Задержка репликации** (в асинхронных системах): вторичные узлы могут отставать → **временная несогласованность**

Минусы

- **Задержка репликации** (в асинхронных системах): вторичные узлы могут отставать → **временная несогласованность**
- **Дополнительное использование ресурсов**: каждый узел хранит полную копию данных своего шарда

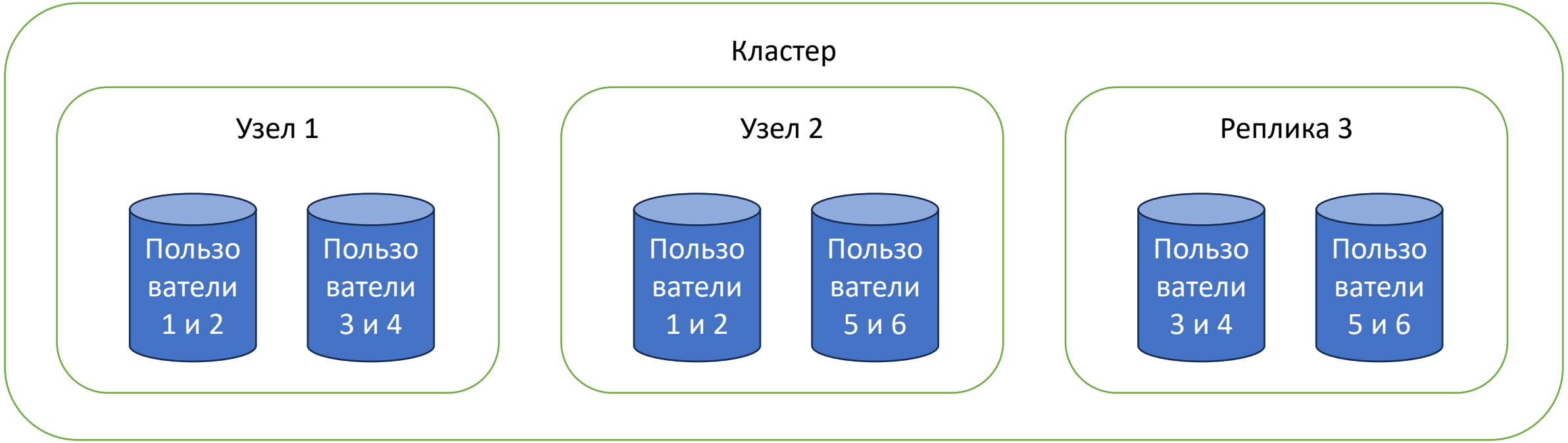
Шардирование

Разделение данных на части (шарды)



Комбинируем

Всегда есть возможность совмещать оба метода масштабирования в зависимости от данных и требований к доступности, скорости и сохранности



Плюсы

Плюсы

- **Масштабируемость**: можно добавлять новые шарды по мере роста данных

Плюсы

- **Масштабируемость**: можно добавлять новые шарды по мере роста данных
- **Производительность**: запросы обрабатываются параллельно на разных узлах

Плюсы

- **Масштабируемость**: можно добавлять новые шарды по мере роста данных
- **Производительность**: запросы обрабатываются параллельно на разных узлах
- **Хранение больших объёмов**: нет ограничений одного диска или сервера

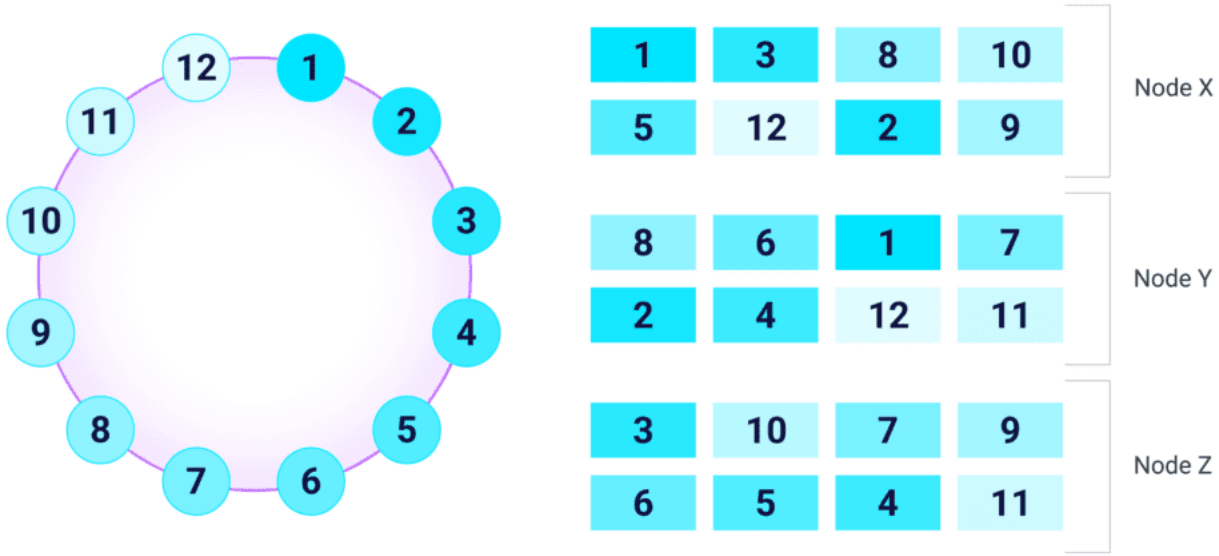
Минусы

Минусы

- **Сложность запросов**: нельзя легко делать JOIN’ы между шардами

Минусы

- **Сложность запросов**: нельзя легко делать JOIN’ы между шардами
- **Неравномерная нагрузка**: если ключ шардирования выбран плохо (например, все данные попадают в один шард — «hot spot»)



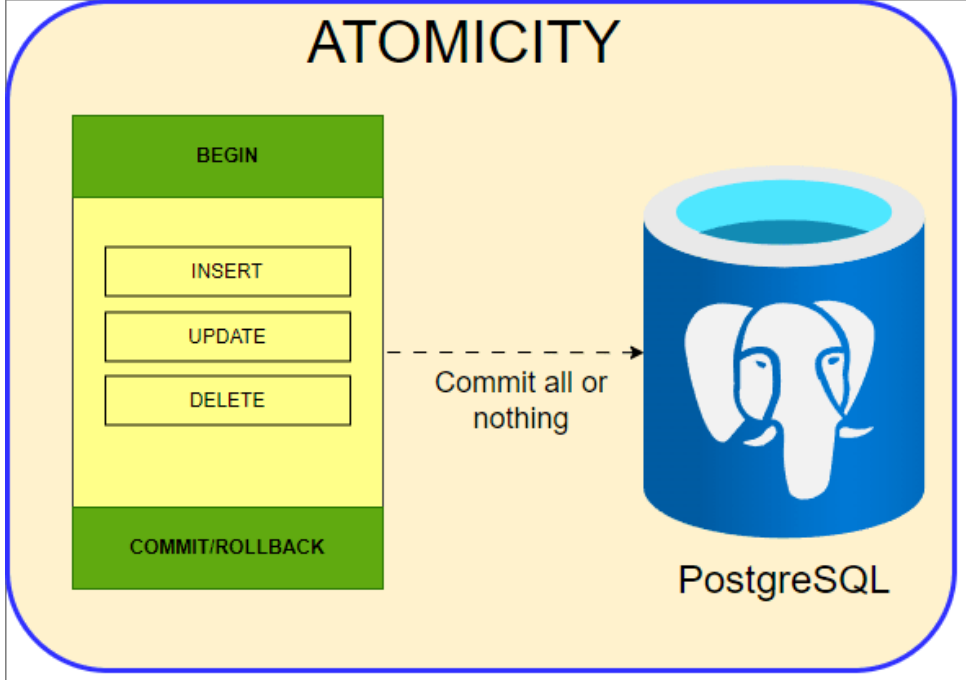
Минусы

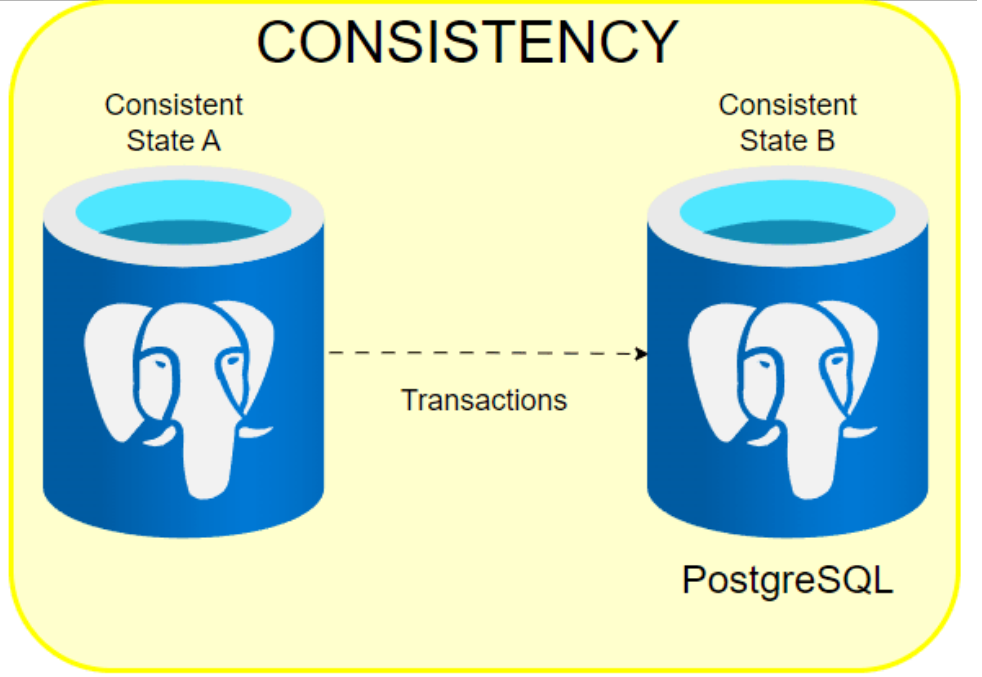
- **Сложность запросов**: нельзя легко делать JOIN’ы между шардами
- **Неравномерная нагрузка**: если ключ шардирования выбран плохо (например, все данные попадают в один шард — «hot spot»)
- **Управление сложнее**: нужно отслеживать, где какие данные

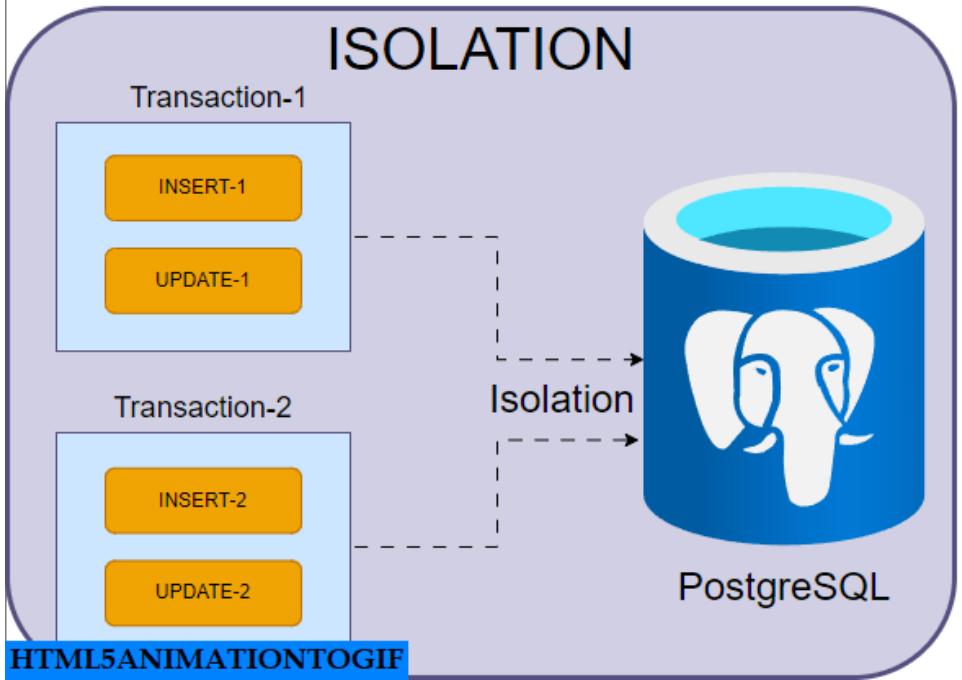
Основные особенности NoSQL

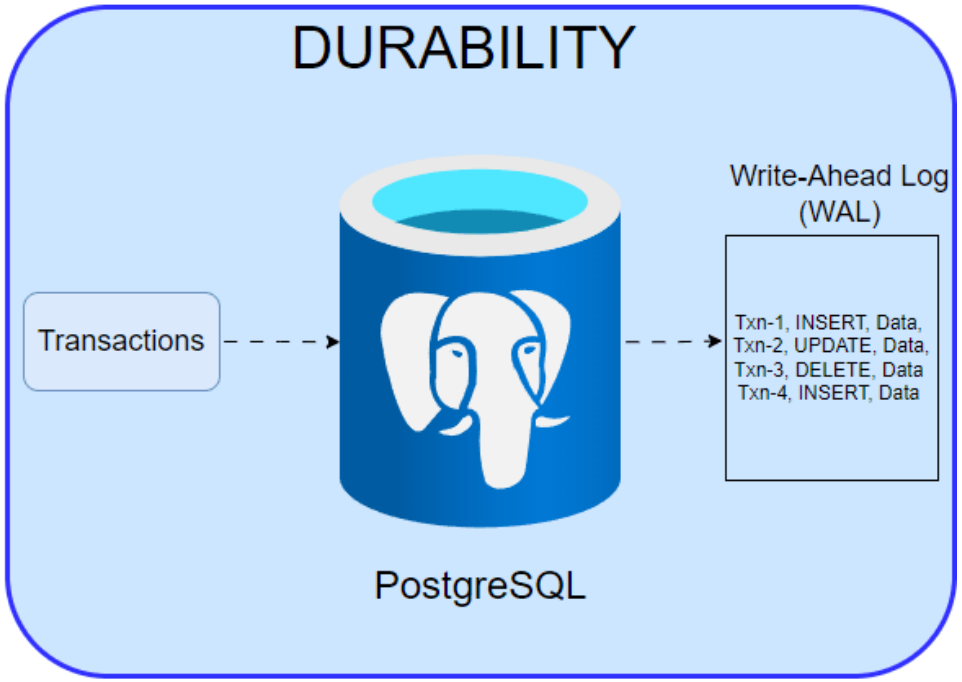
- Большой выбор баз данных в зависимости от задачи
- Гибкая схема данных
- Высокая масштабируемость
- Обработка большого числа запросов и снижение требований к ACID

ACID???

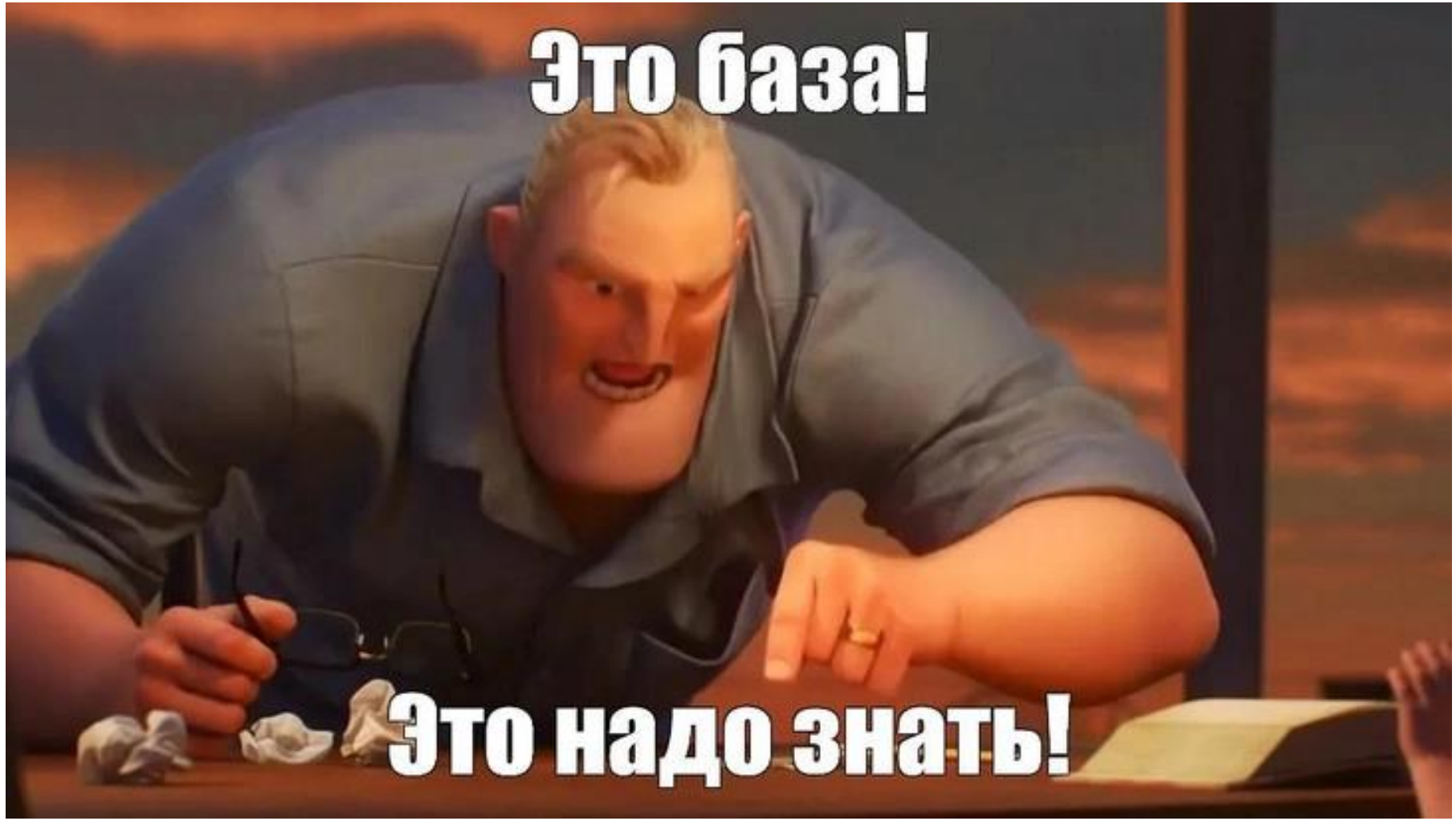

ATOMICITY
BEGIN
INSERT
UPDATE
DELETE
COMMIT/ROLLBACK
Commit all or nothing
PostgreSQL


CONSISTENCY
Consistent State A
Consistent State B
Transactions
PostgreSQL

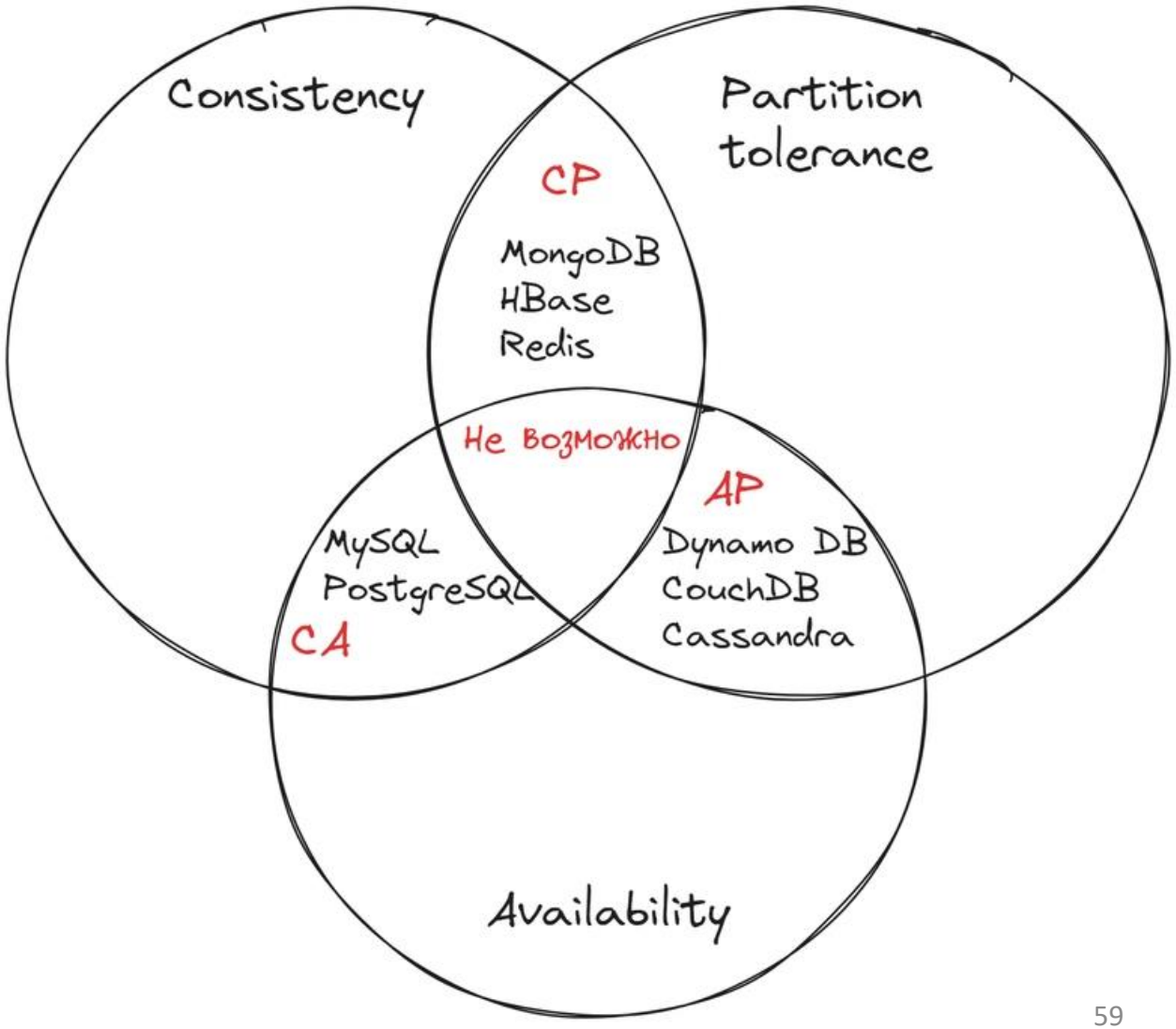

ISOLATION
Transaction-1
INSERT-1
UPDATE-1
Transaction-2
INSERT-2
UPDATE-2
Isolation
PostgreSQL
HTML5ANIMATIONTOGIF


DURABILITY
Transactions
Write-Ahead Log (WAL)
Txn-1, INSERT, Data,
Txn-2, UPDATE, Data,
Txn-3, DELETE, Data
Txn-4, INSERT, Data
PostgreSQL

BASE



C — Consistency (Согласованность)
A — Availability (Доступность)
P — Partition tolerance (Устойчивость к сетевым разделениям)



BASE

BA - **Basically Available**

Система **всегда отвечает на запросы**, даже если часть узлов недоступна

BASE

BA - **Basically Available**

Система **всегда отвечает на запросы**, даже если часть узлов недоступна

S - **Soft state**

Состояние системы **может меняться со временем**, даже без входящих запросов (например, из-за фоновой репликации)

BASE

BA - **Basically Available**

Система **всегда отвечает на запросы**, даже если часть узлов недоступна

S - **Soft state**

Состояние системы **может меняться со временем**, даже без входящих запросов (например, из-за фоновой репликации)

E - **Eventually consistent**

Система **не гарантирует мгновенной согласованности**, но **со временем все узлы придут к одному состоянию**

Что меняется при выборе BASE

Что меняется при выборе BASE

- Меньше гарантий: слабая изоляция, возможна временная несогласованность данных

Что меняется при выборе BASE

- Меньше гарантий: слабая изоляция, возможна временная несогласованность данных
- Система остаётся доступной даже при частичных сбоях и легко масштабируется горизонтально

Что меняется при выборе BASE

- Меньше гарантий: слабая изоляция, возможна временная несогласованность данных
- Система остаётся доступной даже при частичных сбоях и легко масштабируется горизонтально
- Консистентность «в итоге», а не «всегда и сразу»

Что меняется при выборе BASE

- Меньше гарантий: слабая изоляция, возможна временная несогласованность данных
- Система остаётся доступной даже при частичных сбоях и легко масштабируется горизонтально
- Консистентность «в итоге», а не «всегда и сразу»
- Дополнительная сложность для разработчиков: необходимо продумывать обработку конфликтов и расхождений

Когда использовать NoSQL базу?

Когда использовать NoSQL базу?

- Требуется высокая производительность

Когда использовать NoSQL базу?

- Требуется высокая производительность
- Требуется возможность «добавить базе ресурсов» на лету

Когда использовать NoSQL базу?

- Требуется высокая производительность
- Требуется возможность «добавить базе ресурсов» на лету
- Готовы проектировать таблицу под запрос

Когда использовать NoSQL базу?

- Требуется высокая производительность
- Требуется возможность «добавить базе ресурсов» на лету
- Готовы проектировать таблицу под запрос
- Простые запросы на чтение и запись (не говорим про аналитические базы, про них позже)

Когда NoSQL базы не подойдут?

Когда NoSQL базы не подойдут

- Нужны сложные транзакции с жёсткой консистентностью (пример - финансовые операции)

Когда NoSQL базы не подойдут

- Нужны сложные транзакции с жёсткой консистентностью (пример - финансовые операции)
- Много сложных запросов с JOIN по разным сущностям

Когда NoSQL базы не подойдут

- Нужны сложные транзакции с жёсткой консистентностью (пример - финансовые операции)
- Много сложных запросов с JOIN по разным сущностям
- Важна строгая и стабильная схема, сильные требования к отчётности и аудитам

Когда NoSQL базы не подойдут

- Нужны сложные транзакции с жёсткой консистентностью (пример - финансовые операции)
- Много сложных запросов с JOIN по разным сущностям
- Важна строгая и стабильная схема, сильные требования к отчётности и аудитам
- Команда не готова поддерживать сложную логику целостности на уровне приложения

Перерыв

Scylla

Особенности

- Аналог cassandra (полностью совестимы), но пошустрее (написана на C++)
- Запросы пишутся на CQL – аналог SQL
- Сама балансирует данные как при увеличении количества данных, так и при добавлении новых нод
- Имеет кольцевую архитектуру (masterless)
- Высокая пропускная способность, низкие задержки

Scylla

Примеры использования:

- Хранение динамических настроек
- Логи, метрики
- Уведомления
- Кэши и т.д.

Архитектура

- Кластер Scylla – набор равноправных соединенных сетью узлов

Архитектура

- Кластер Scylla – набор равноправных соединенных сетью узлов
- Данные располагаются на узлах – нодах. Каждая нода – отдельный экземпляр ScyllaDB

Архитектура

- Кластер Scylla – набор равноправных соединенных сетью узлов
- Данные располагаются на узлах – нодах. Каждая нода – отдельный экземпляр ScyllaDB
- Ресурсы (cpu, I/O, память) каждой ноды разбиты на шарды – каждый шард работает с данными независимо

Архитектура

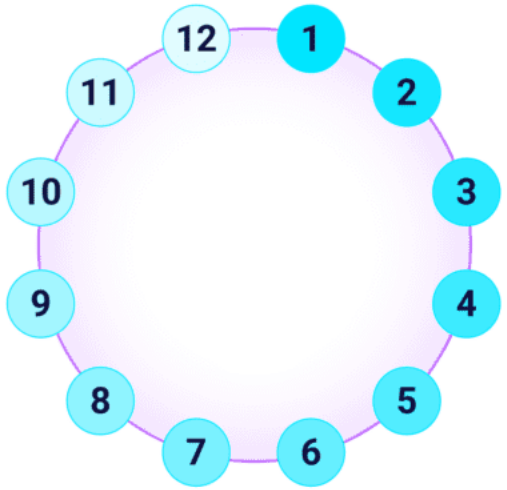
- Кластер Scylla – набор равноправных соединенных сетью узлов
- Данные располагаются на узлах – нодах. Каждая нода – отдельный экземпляр ScyllaDB
- Ресурсы (cpu, I/O, память) каждой ноды разбиты на шарды – каждый шард работает с данными независимо
- Wide-column store – данные в таблицах организованы в партиции, внутри которых строки с одинаковым partition key, отсортированные по clustering key

Архитектура

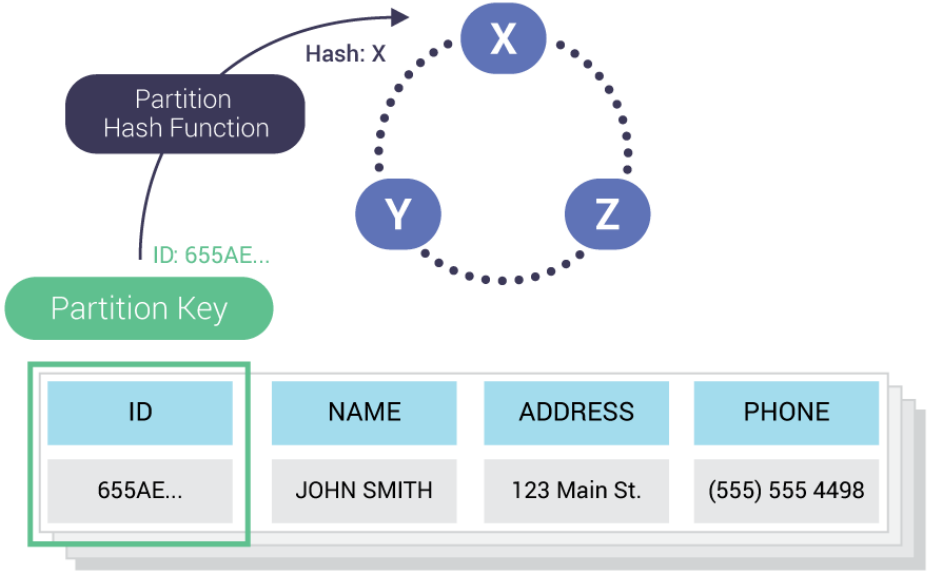
- Кластер Scylla – набор равноправных соединенных сетью узлов
- Данные располагаются на узлах – нодах. Каждая нода – отдельный экземпляр ScyllaDB
- Ресурсы (cpu, I/O, память) каждой ноды разбиты на шарды – каждый шард работает с данными независимо
- Wide-column store – данные в таблицах организованы в партиции, внутри которых строки с одинаковым partition key, отсортированные по clustering key
- Партиции могут объединяться в tablets (с версии 5.0) – логические диапазоны данных, которые быстро перемещаются между нодами при балансировке

Архитектура

- Токены представляются в виде кольца
- Каждая нода содержит определенное число токенов (в зависимости от фактора репликации)
- При чтении/вставке от ключа партиции вычисляется hash-функция



1	3	8	10	Node X
5	12	2	9	
8	6	1	7	Node Y
2	4	12	11	
3	10	7	9	Node Z
6	5	4	11	



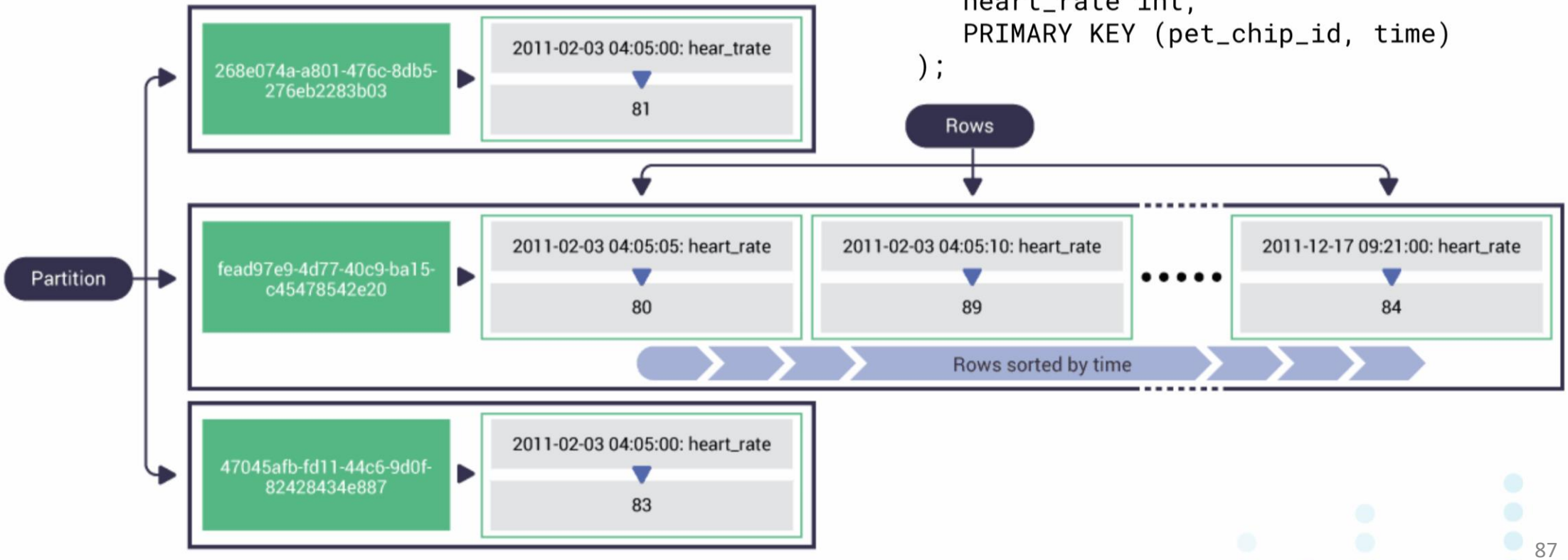
Хранение данных

*Таблицы объединяются в Keyspace – аналог database из реляционных баз

Partitions and rows

Clustering key

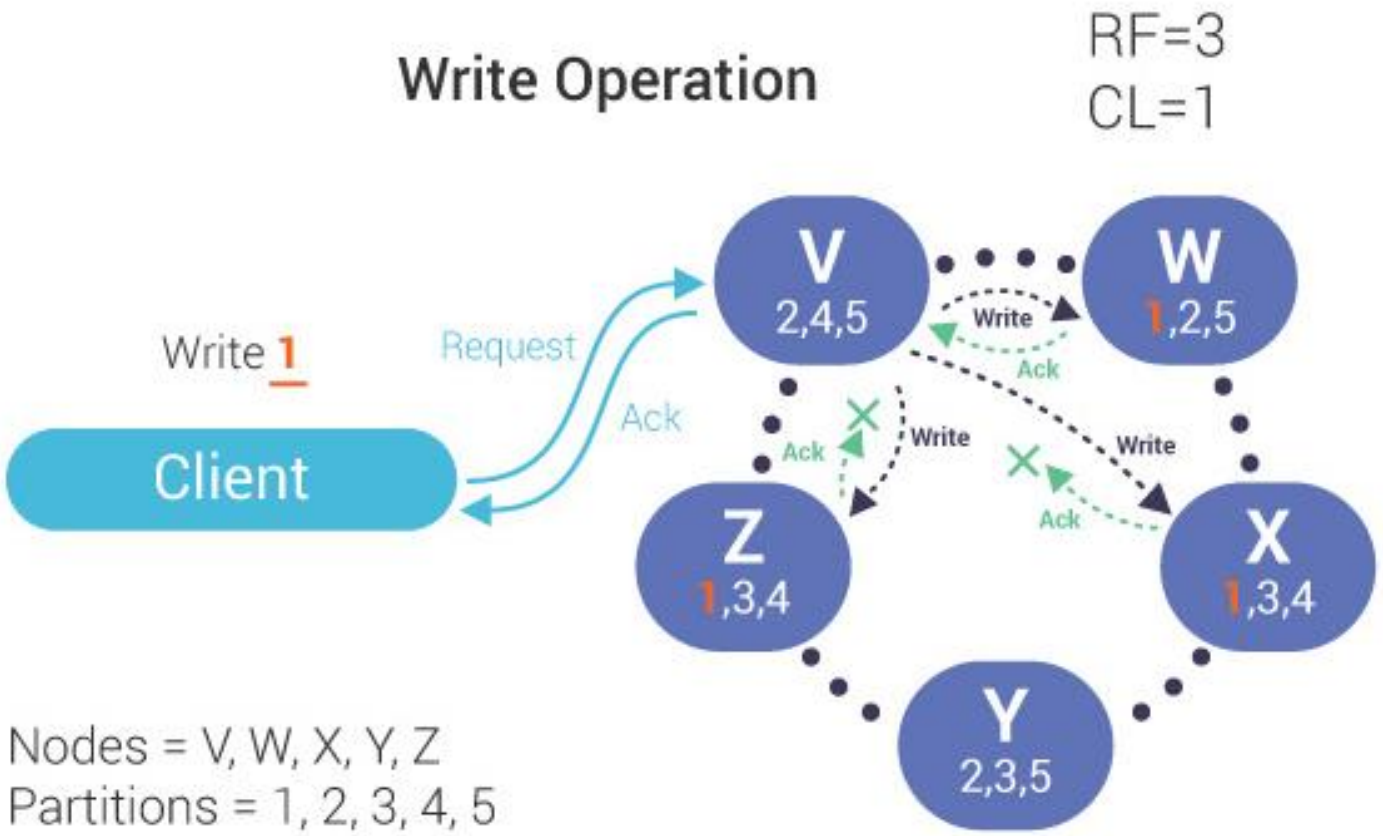
```
CREATE TABLE heartrate_v1 (
   pet_chip_id uuid,
   time timestamp,
   heart_rate int,
   PRIMARY KEY (pet_chip_id, time)
);
```



Переменная согласованность

*В зависимости от уровня согласованности можно варьировать систему от AP к CP

- ANY - любая
- ONE/TWO/THREE – ближайшие 1/2/3
- ALL – все

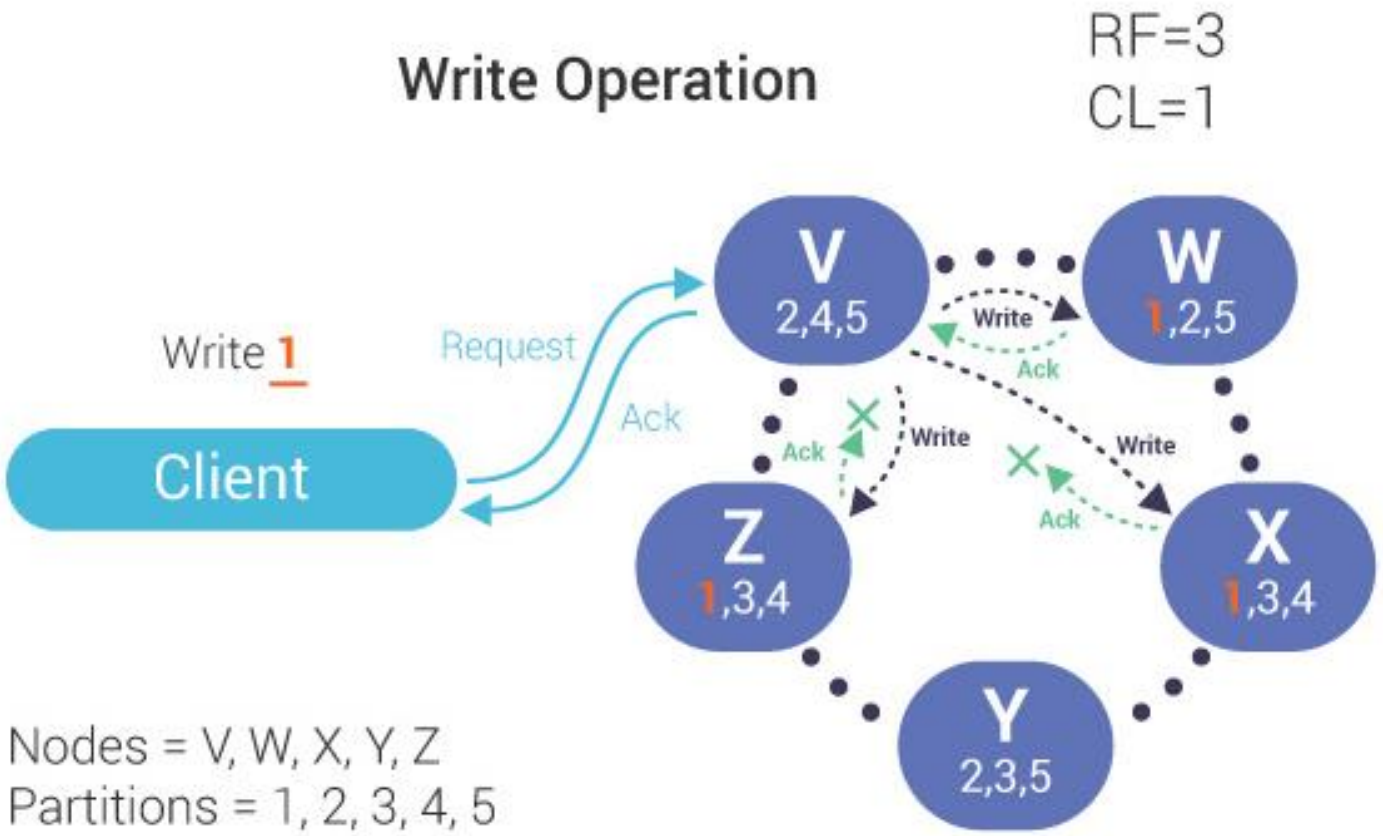


Переменная согласованность

*В зависимости от уровня согласованности можно варьировать систему от AP к CP

- ANY - любая
- ONE/TWO/THREE – ближайшие 1/2/3
- ALL – все

- QUORUM – большинство ((RF / 2) + 1)
- LOCAL_QUORUM – quorum в рамках 1 ДЦ
- EACH_QUORUM – quorum в каждом ДЦ
- LOCAL_ONE – 1 в том же ДЦ

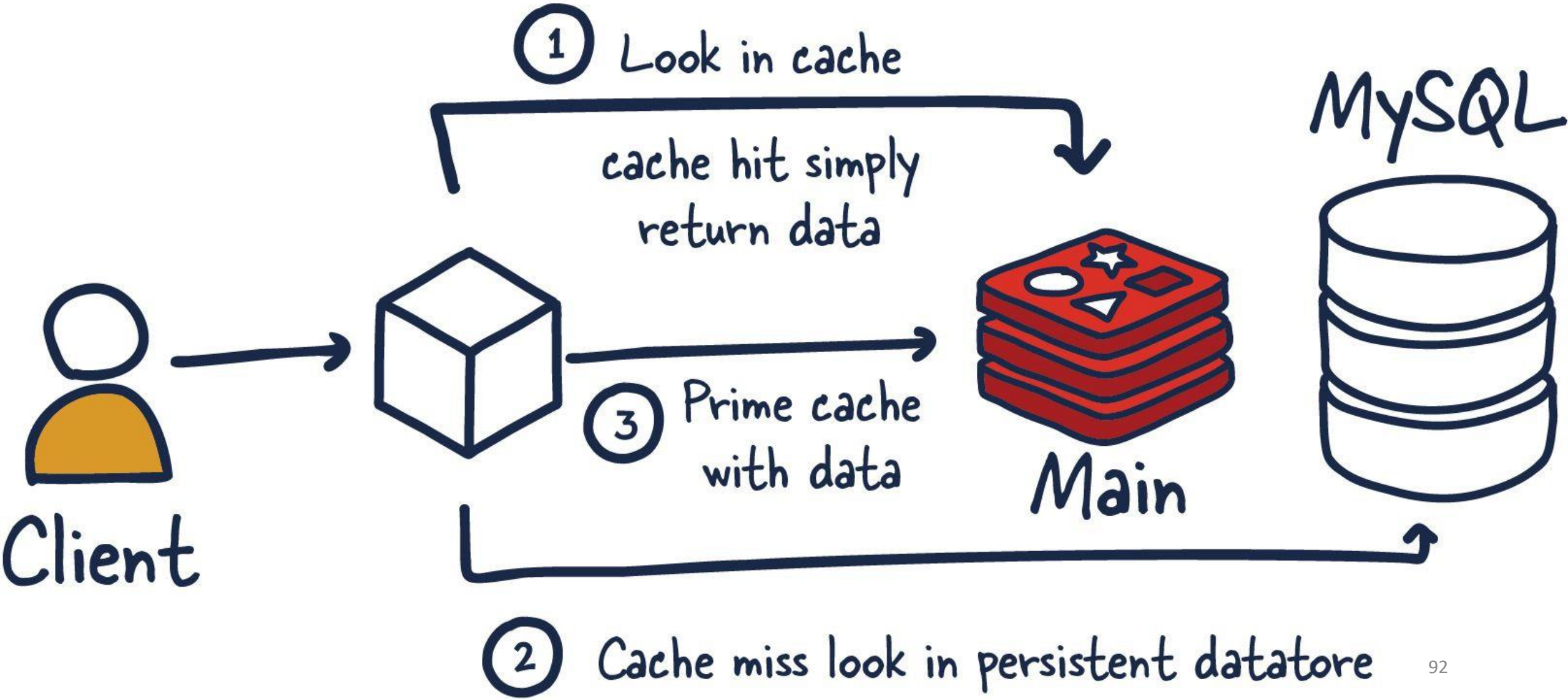


Посмотрим на практике

Redis

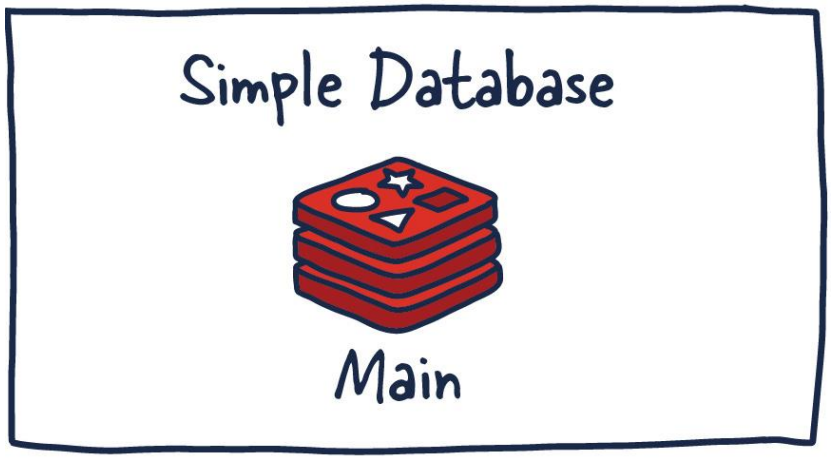
- Хранилище «Ключ-значение» с открытым исходным кодом
- Хранит данные в оперативной памяти
- Однопоточный
- Используется как кэш, база данных, брокер сообщений
- Имеет встроенные механизмы репликации
- Может сохранять данные на диск

How is redis traditionally used



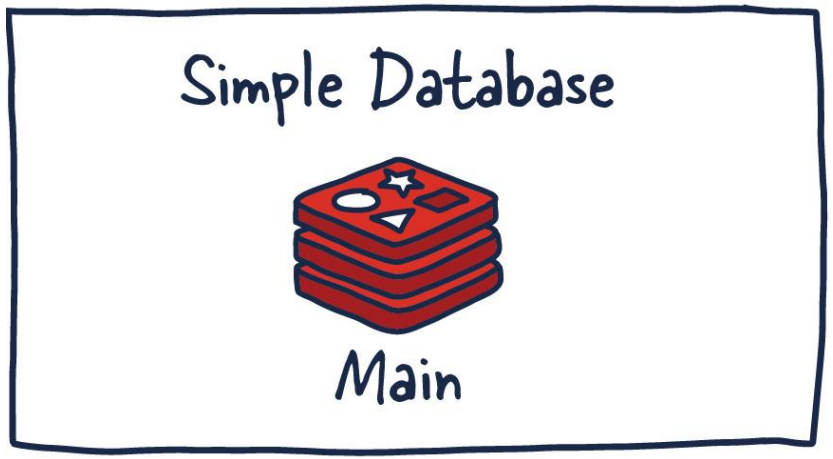
Одиночный экземпляр Redis (базовый минимум)

- Самый простой вариант развертывания
- Если памяти и cpu достаточно – этого варианта может быть вполне достаточно
- Если инстанс упадет – то (а что будет?)



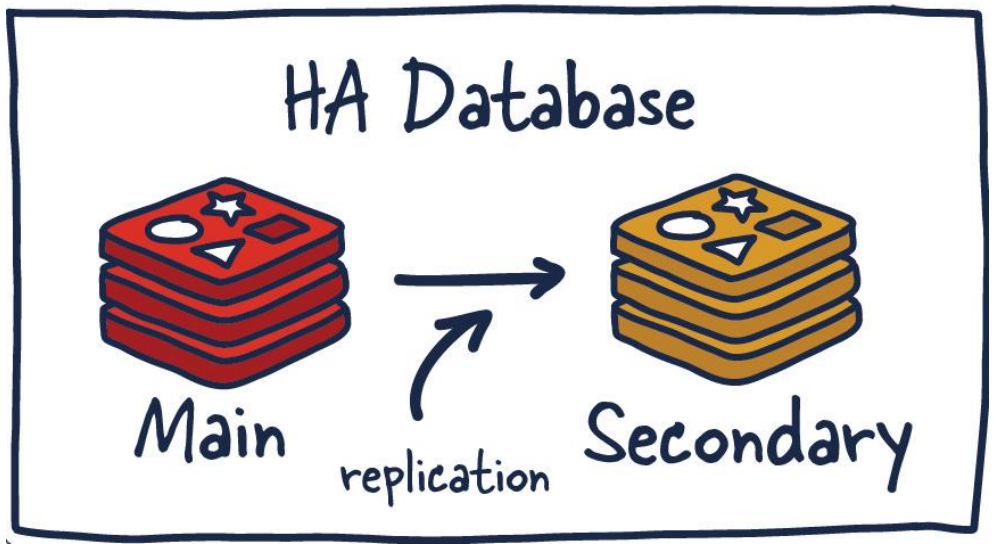
Одиночный экземпляр Redis

- Самый простой вариант развертывания
- Если памяти и cpu достаточно – этого варианта может быть вполне достаточно
- Если инстанс упадет – то
  - либо случится даунтайм в случае использование в виде базы
  - либо резко возрастет нагрузка на основную базу



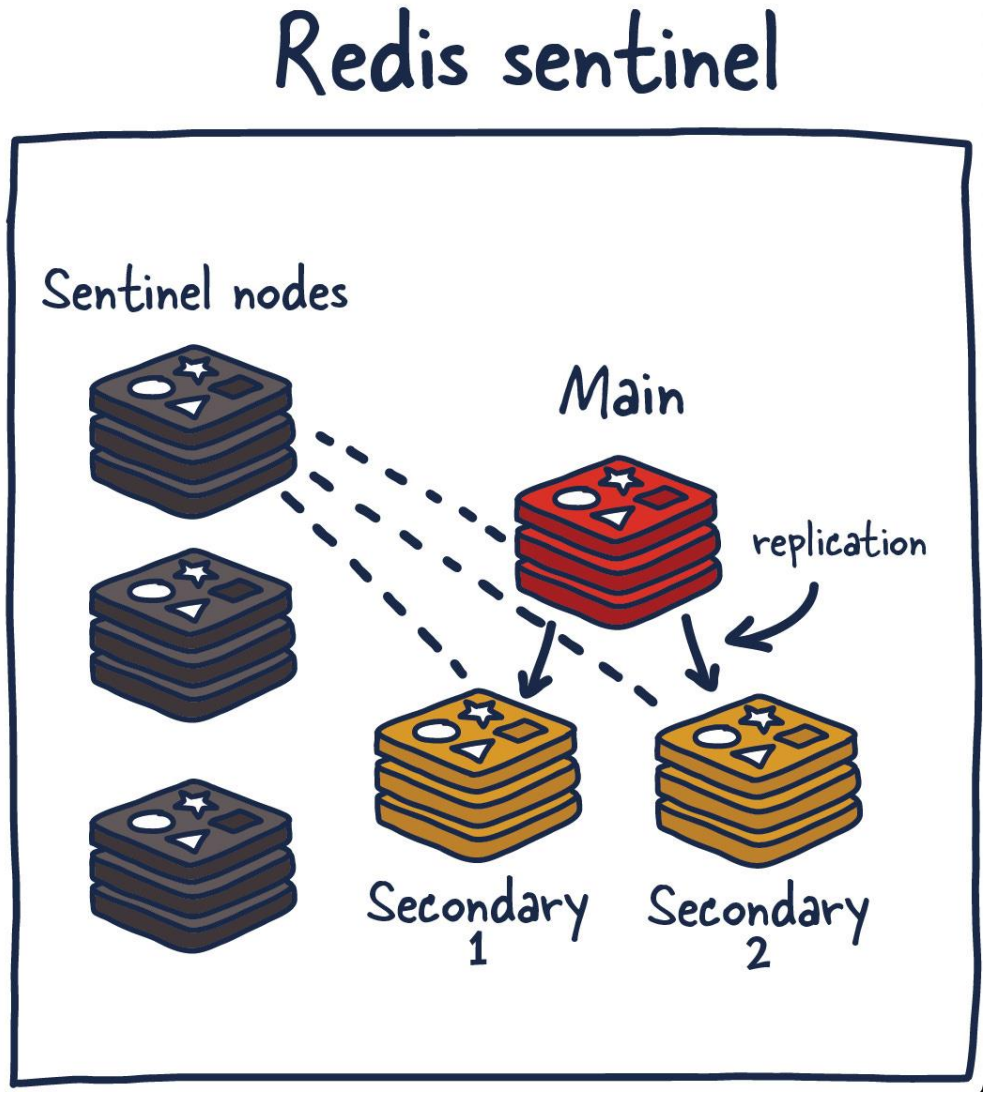
HA (High availability)

- Горизонтальное масштабирование
- Есть мастер и реплики
- Мастер досылает изменения, ориентируясь на Replication id и offset
- Если отставания слишком большое, происходит полная синхронизация



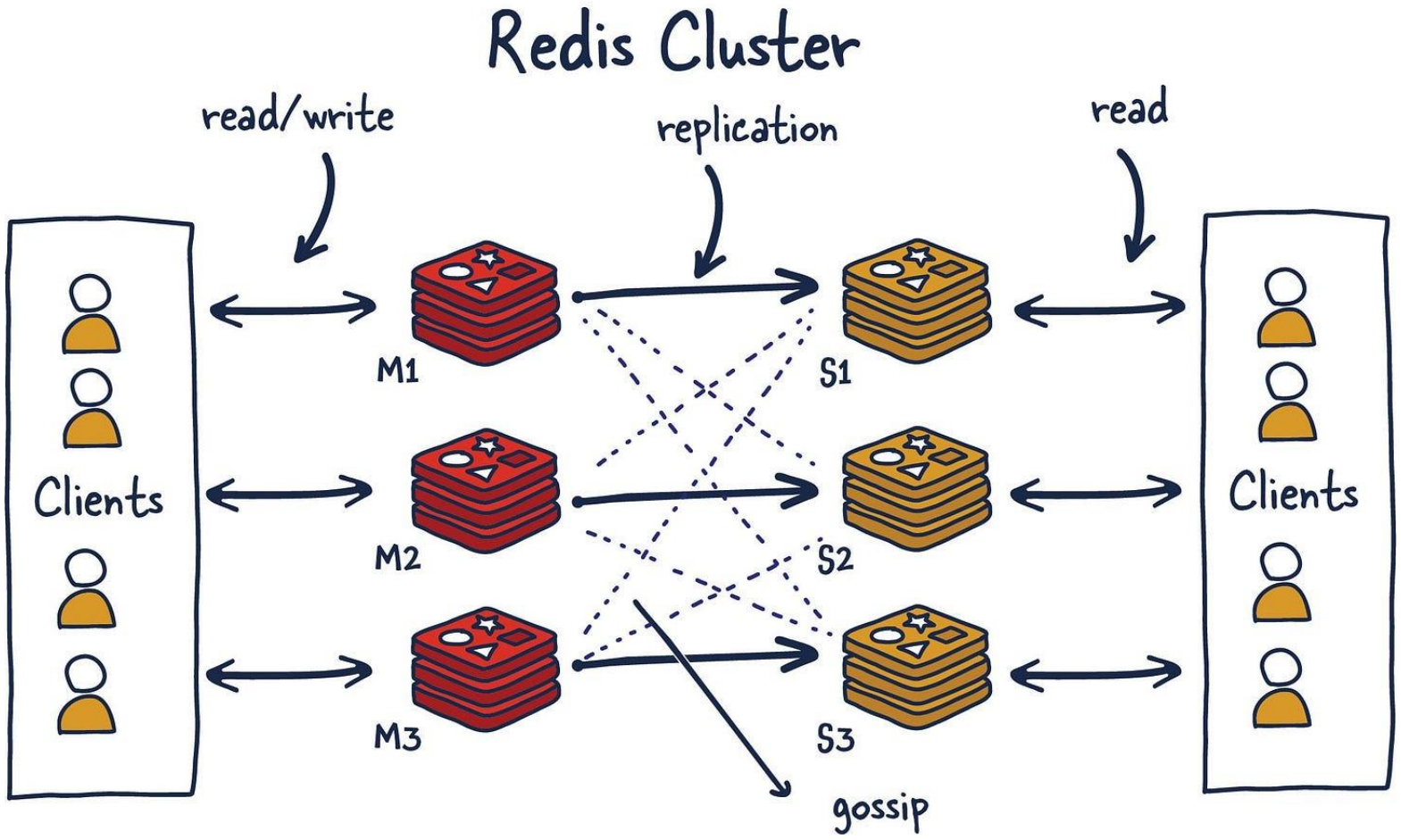
Sentinel

- Та же структура master + replica, но добавляются sentinel ноды (минимум 3 для кворума)
- Sentinel ноды:
  - Мониторят, жив ли мастер
  - Уведомляют от проблеме
  - Выбирают новый мастер, если старый сломался
  - Обнаруживают и направляют новую ноду при подключении



Cluster (роскошный максимум)

- Шардирование
- Механизм обмена информацией (gossiping)
- Сам определяет, какие мастер-ноды вышли из строя, и заменяет их



RPM (Redis Persistent Models)

- RDB (Redis Database) – снимки памяти в файлы на жестком диске
- AOF (Append Only File) – постоянно логирует каждую операцию записи, которую получает сервер
- Можно комбинировать – если нужна надежность и готовы пожертвовать скоростью

Посмотрим на практике

OLTP vs OLAP

OLTP – online transaction processing

- Короткие, простые запросы: INSERT, UPDATE, DELETE, SELECT по ключу
- Высокая частота: тысячи/миллионы запросов в секунду
- Низкая задержка: ответ за миллисекунды
- Данные, меняющиеся в реальном времени

Написал запрос, который выполняется 10 секунд? По-твоему это долго?

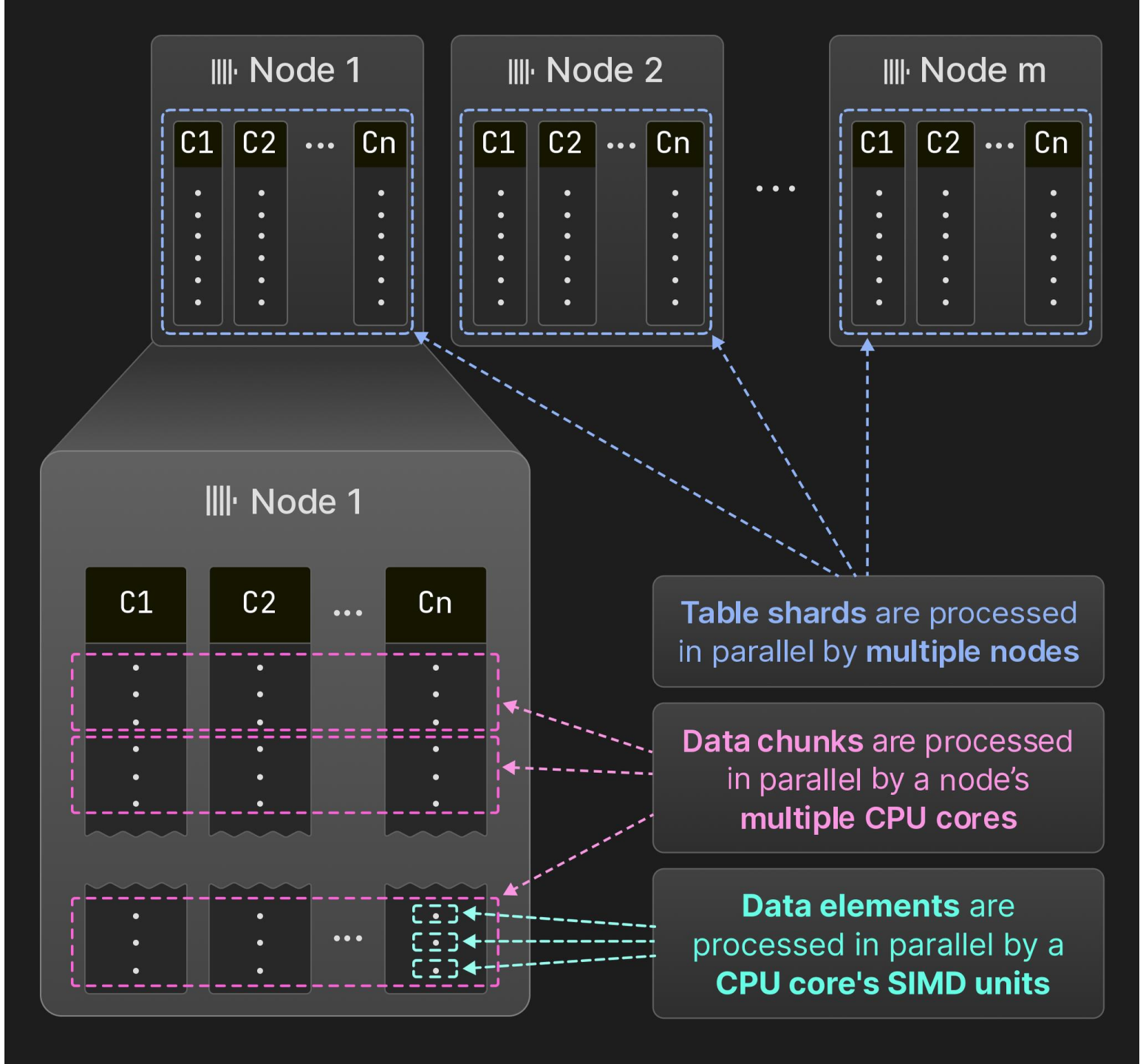
OLTP vs OLAP

OLAP – online analytical processing

- Сложные, долгие запросы: агрегация, JOIN’ы, группировка по множеству полей
- Низкая частота: десятки/сотни запросов в час
- Высокая пропускная способность: сканирование миллионов/миллиардов строк
- Основная нагрузка при чтении
- Исторические данные: месяцы или годы

ClickHouse

- Колоночная база данных -> позволяет читать только нужные столбцы
- Написана на C++
- Очень эффективно сжимает данные (дорого по CPU)
- Параллельная обработка больших запросов на нескольких ядрах
- Распределенная обработка запросов на разных шардах
- Поддерживает SQL
- Первичные индексы + вторичные индексы (индексы пропуска данных)


Node 1
C1
C2
...
Cn
Node 2
C1
C2
...
Cn
...
Node m
C1
C2
...
Cn
Node 1
C1
C2
...
Cn
...
Table shards are processed in parallel by multiple nodes
Data chunks are processed in parallel by a node's multiple CPU cores
Data elements are processed in parallel by a CPU core's SIMD units

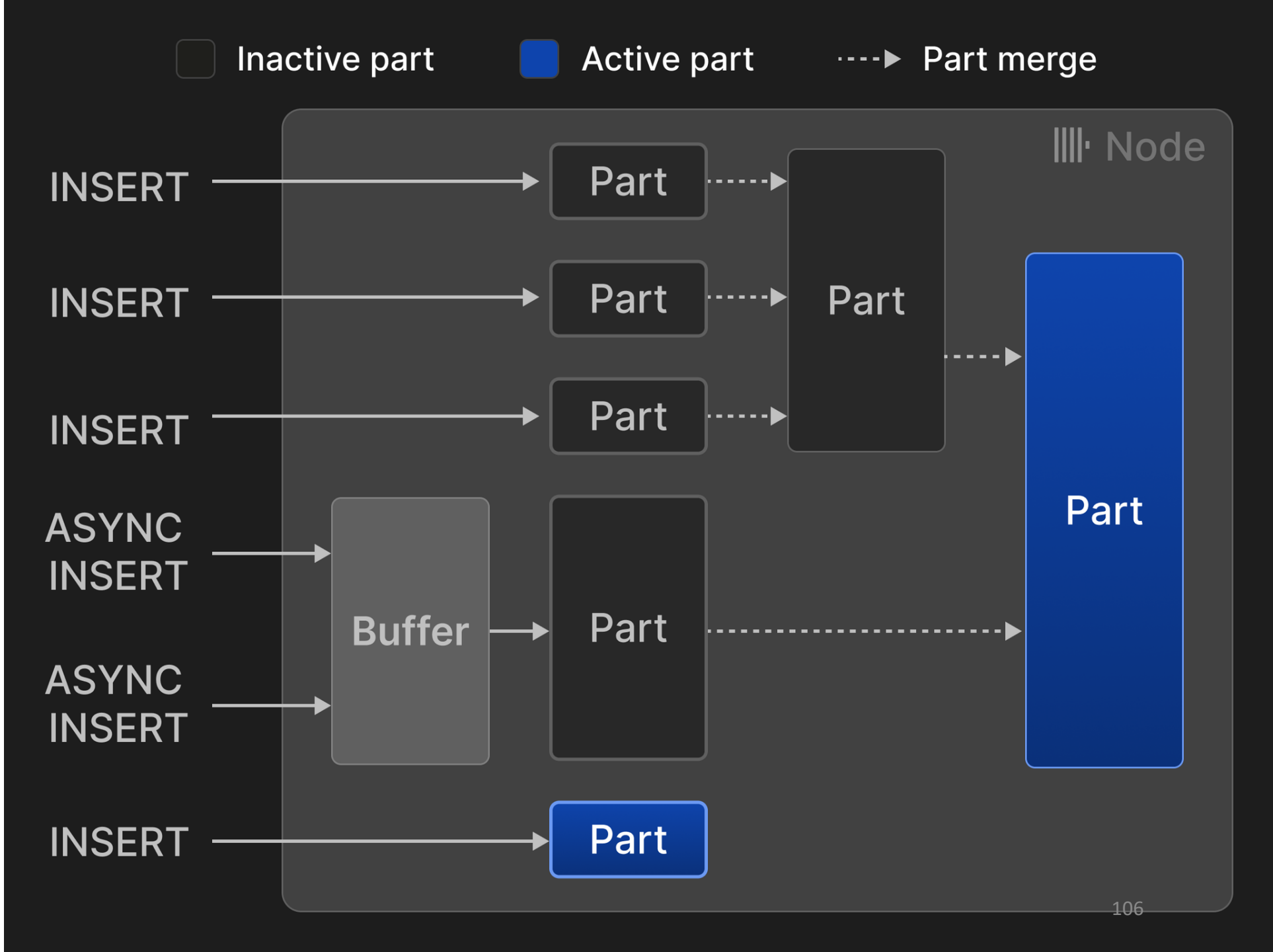
А в чем особенность для пользователя?

- Движок таблицы: MergeTree, Log и их производные
  - как и где хранятся данные
  - какие запросы поддерживаются
  - реплицируются данные или нет
- Есть рекомендации по вставке данных
- Огромное количество форматов данных и готовых функций работы с ними

```
CREATE DATABASE IF NOT EXISTS helloworld

CREATE TABLE helloworld.my_first_table
(
    user_id UInt32,
    message String,
 DateTime DateTime,
    metric Float32
)
ENGINE = MergeTree()
PRIMARY KEY (user_id, timestamp)
ORDER BY (DateTime);
```

Как пишутся данные



Подробнее [про архитектуру]

Hadoop

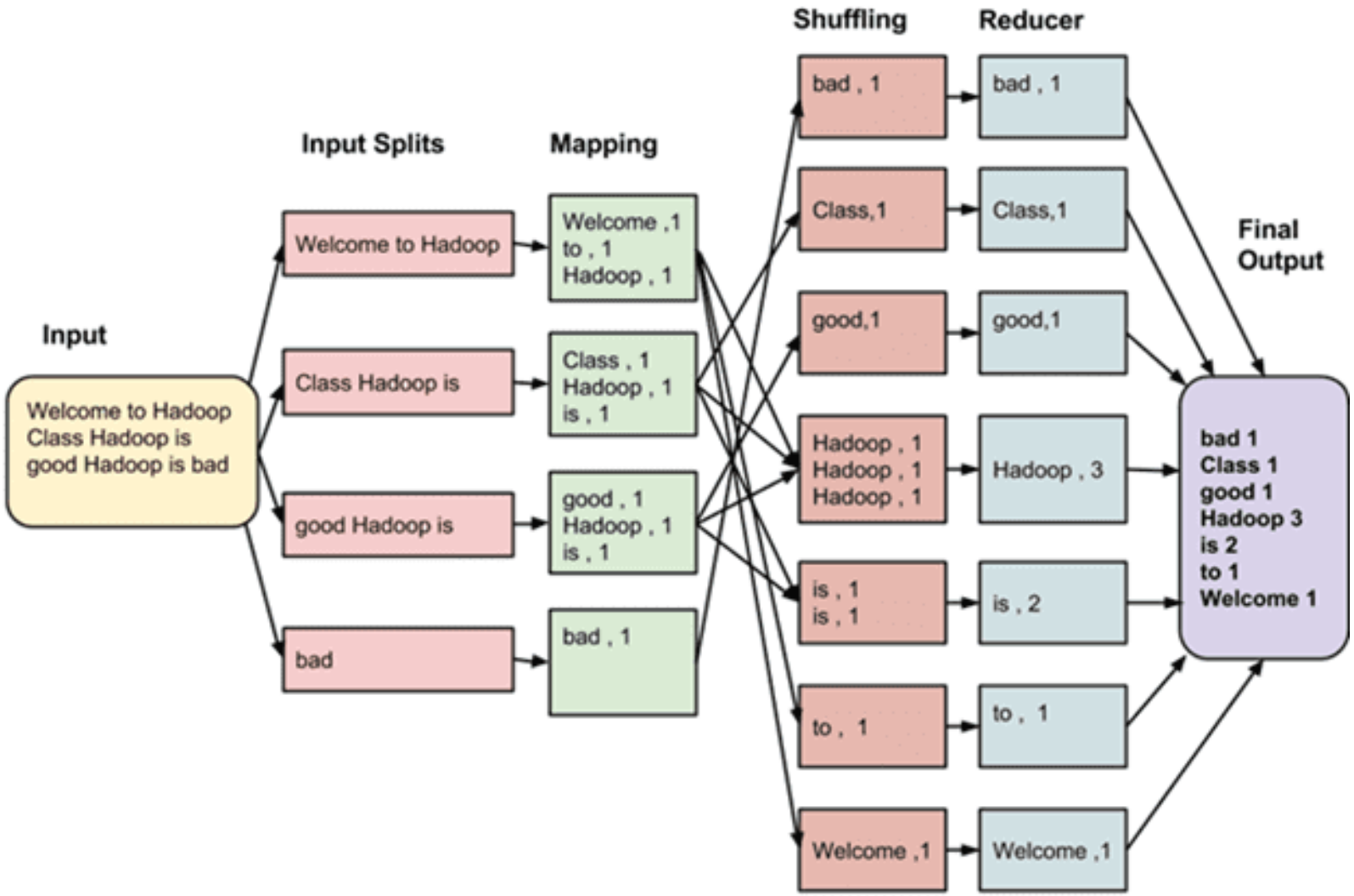
Hadoop – не база данных, а набор инструментов для распределенного хранение и обработки big data

- HDFS – распределенное файловое хранилище, некий жесткий диск, расположенный физически на разных машинах
- YARN – менеджер управления ресурсами в кластере
- Hive – СУБД на базе Hadoop, позволяет писать запросы в SQL-подобном виде
- Различные клиенты – zeppelin и производные (я использую querybook)
- MapReduce – модель параллельной обработки данных

MapReduce

Идея в том, чтобы разбить большую задачу на мелкие и распараллелить

- Map – обрабатывает данные и отдает пары ключ-значение
- Shuffle – группирует по ключу
- Reduce – объединяет в итоговый результат



Литература

[Архитектура Scylla](#)
[Scylla docs](#)

[Архитектура Redis](#)
[Redis docs](#)

[Архитектура Clickhouse](#)
[Clickhouse docs](#)

[Репозиторий с примерами](#)

Вопросы?

Спасибо за внимание!

Обратная связь по лекции